

[redacted] – Web Application Security

[redacted]

CA3 Project (50%)

Technical Report

Group: 2

Members:

Mark Diadyk – [redacted]

Maël Conejos Crusson – [redacted]

Joseph Cronin – [redacted]

Ken Sweeney – [redacted]

[redacted]

Contents

School of Informatics & Cybersecurity - Assessment Submission Coversheet [Mark] 2
School of Informatics & Cybersecurity - Assessment Submission Coversheet [Maël] 3
School of Informatics & Cybersecurity - Assessment Submission Coversheet [Joseph] 4
School of Informatics & Cybersecurity - Assessment Submission Coversheet [Ken] 5
Completed Challenges 6
Individual Contributions 7
Vulnerability Analysis 12
Injection [Joseph] 12
Broken Authentication [Joseph] 14
Sensitive Data Exposure [Mark] 18
Improper Input Validation [Mark] 20
Broken Access Control [Maël] 22
Observability Failures [Maël] 24
Cryptographic Issues [Ken] 26
Security Misconfiguration [Ken] 30
SAST Testing 32
Testing 32
The Findings 33
Analysis 33
DAST Testing 38
Testing 38
The Findings 39
Analysis 40
Remediation Recommendations 45
DAST Conclusion 45
References 46

School of Informatics & Cybersecurity - Assessment Submission Coversheet [Mark]

Student Information

Student Number:	[redacted]
Student Name:	Mark Diadyk

Module Information

Module Title:	Web Application Security
Module Code:	[redacted]

Assessment Information

Assessment Title:	CA3 Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

[redacted]

☑ I have completed the TU Dublin AI Driving License theory test.

[redacted]

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	[redacted]
Date:	27.04.2026

School of Informatics & Cybersecurity - Assessment Submission Coversheet [Maël]

Student Information

Student Number:	[redacted]
Student Name:	Maël Conejos Crusson

Module Information

Module Title:	Web Application Security
Module Code:	[redacted]

Assessment Information

Assessment Title:	CA3 Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

[redacted]

☑ I have completed the TU Dublin AI Driving License theory test.

[redacted]

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	[redacted]
Date:	27/04/2026

School of Informatics & Cybersecurity - Assessment Submission Coversheet [Joseph]

Student Information

Student Number:	[redacted]
Student Name:	Joseph Cronin

Module Information

Module Title:	Web Application Security
Module Code:	[redacted]

Assessment Information

Assessment Title:	CA3 Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

[redacted]

☑ I have completed the TU Dublin AI Driving License theory test.

[redacted]

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	[redacted]
Date:	27/04/2026

School of Informatics & Cybersecurity - Assessment Submission Coversheet [Ken]

Student Information

Student Number:	[redacted]
Student Name:	Ken Sweeney

Module Information

Module Title:	Web Application Security
Module Code:	[redacted]

Assessment Information

Assessment Title:	CA3 Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

[redacted]

☑ I have completed the TU Dublin AI Driving License theory test.

[redacted]

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	[redacted]
Date:	27/04/2026

Total Word Count: [9481]

Completed Challenges

OWASP JuiceShop Scoreboard screenshot

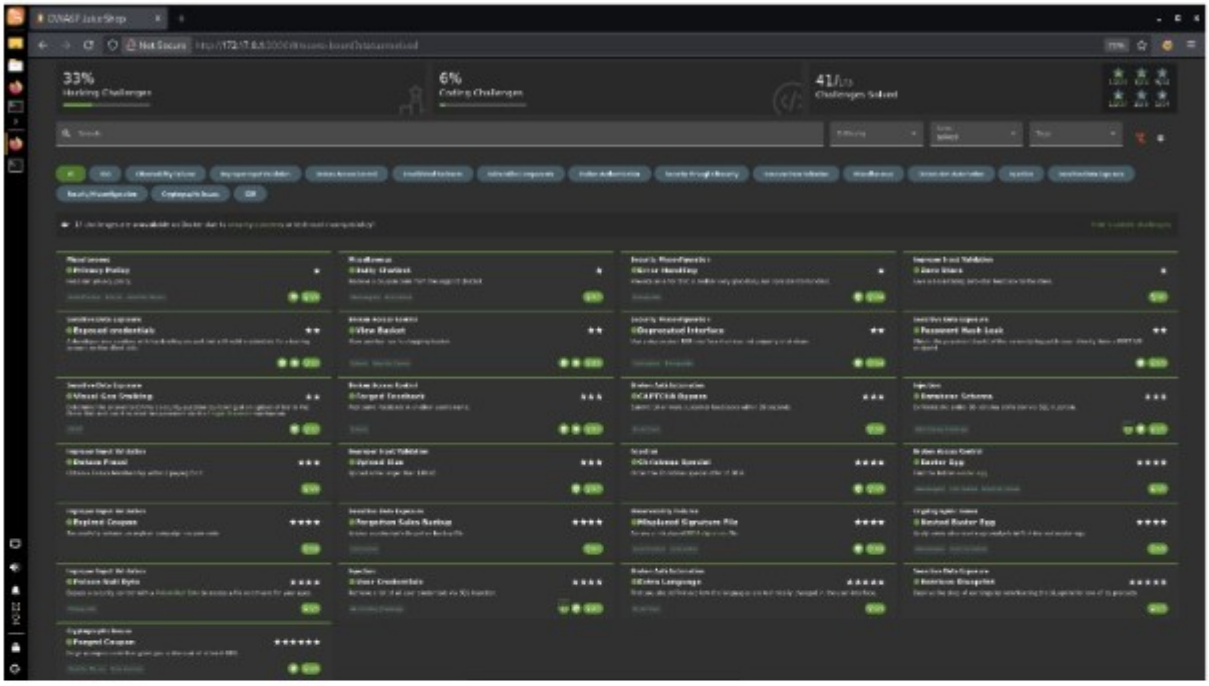


Figure 1: OWASP Juice Shop Completed Challenges #1

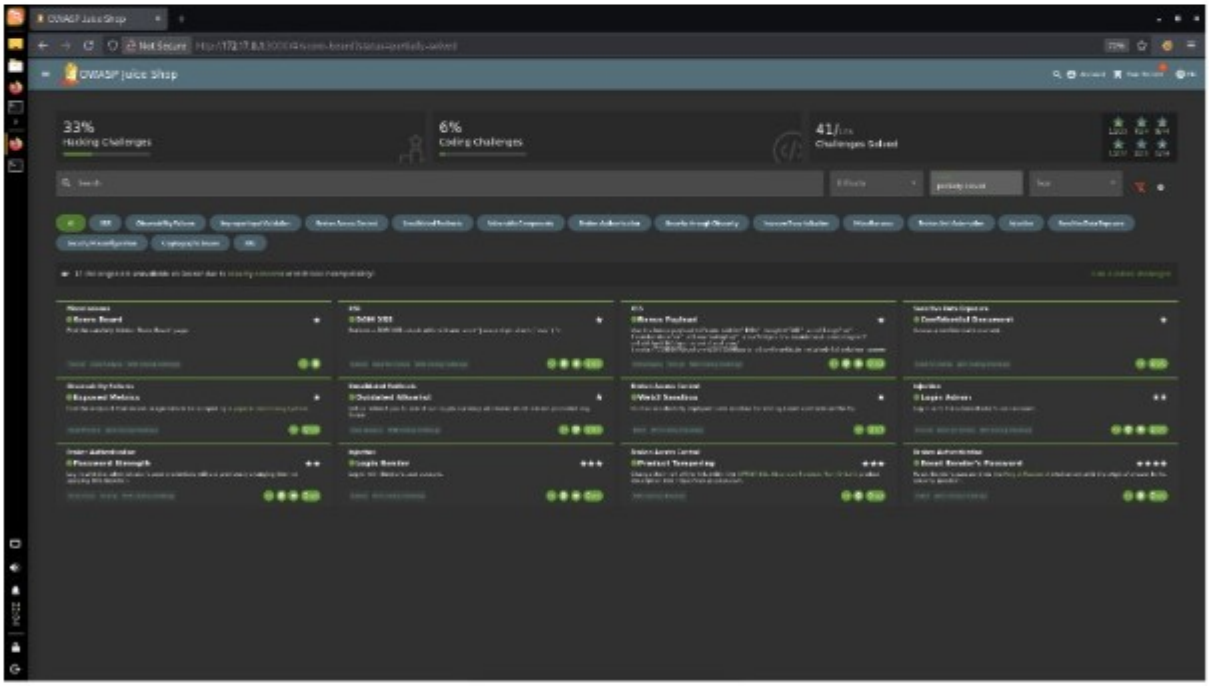
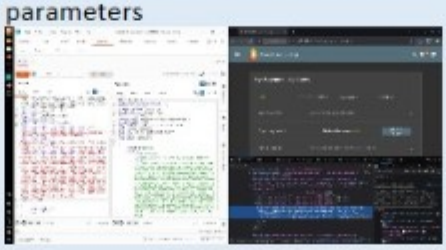
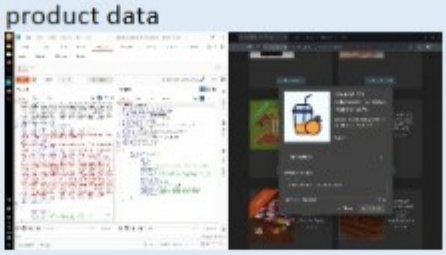
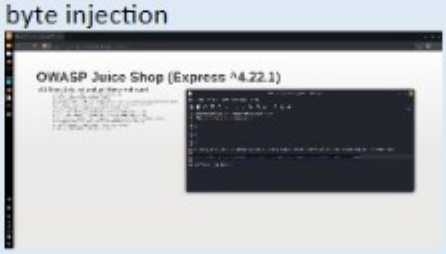


Figure 2: OWASP Juice Shop Completed Challenges #2

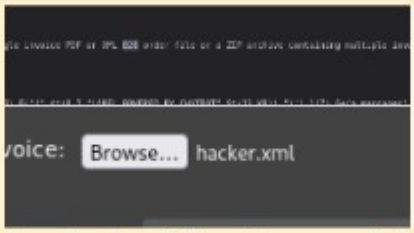
Individual Contributions

Challenge ID	Vulnerability	Student Responsible	Evidence	Mitigation Proposed
Exposed Credentials	**Sensitive Data Exposure**	Mark	 Credentials were discovered in a client-side JavaScript file, main.js, using the browser developer tools, where sensitive login information was hardcoded and accessible to any user.	Do not store/hard-code credentials in user accessible files; use secure storage instead.
Upload Size	**Improper Input Validation**	Mark	 A file larger than 100 KB was successfully uploaded through the complaint form. The Burp Suite request has a large Content-Length value being accepted by the server, confirming a lack of server-side validation.	Server-side validation and enforcement of file size limits.
Database Schema	Injection	Mark	 Using a modified search request in Burp Suite, database schema information, such as table structures, was retrieved, demonstrating that backend queries could be influenced to disclose internal database details.	Use parameterised queries and implement input sanitisation.

User Credentials	Injection	Mark	 Using a manipulated search request in Burp Suite, database schema information, such as table structures, was retrieved, showing that backend queries could be influenced to disclose internal database details.	Use prepared statements and avoid concatenating user input into SQL.
Retrieve Blueprint	**Sensitive Data Exposure**	Mark	 A sensitive blueprint file was accessed directly via a URL without authentication, exposing confidential business data (CAD model).	Restrict access and remove sensitive files from being accessible.
Password Hash Leak	Sensitive Data Exposure	Maël	Challenge solved + modified API request by replacing 'email' field with 'password' 	Use Data Transfer Objects to exclude sensitive fields from API responses + Implement strict schema validation
Deluxe Fraud	Improper Data Validation	Maël	Challenge solved + inspected page to modify button element + modified API request transaction parameters 	Implement strict server-side validation for financial calculations

Product Tampering	**Broken Access Control**	Maël	Challenge solved + execution of unauthorised PUT request to modify product data 	Use Role-Based Access Control on all state-changing API endpoints
Misplaced Signature File	**Observation Failures**	Maël	Challenge solved + access to restricted files in '/ftp' using null byte injection 	Sanitise file path inputs
Extra Language	Broken Anti Automation	Maël	Challenge solved + modified API request file path 'assets/i18n/' to replace current language with new one's code 	Implement allow-list of supported language codes
(Extra) Easter Egg	**Broken Access Control**	Maël	Challenge solved + access to restricted files in '/ftp' using null byte injection 	Sanitise file path inputs
(Extra) Nested Easter Egg	Cryptographic Issues	Maël	Challenge solved + decoded (Base 64) & decrypted (Ceasar Cipher) URL path 	Use encryption rather than obfuscation

View Basket	Broken Access Control	Joseph	Challenge Solved using Burp Suite Intercept and manually changing basket ID before forwarding. This changed the basket being viewed, without logging out the Admin account.	Ensuring that the logged-in user can view only their own basket and that client side manipulation via interception cannot not occur.
Password Strength	**Broken Authentication (Authentication Failures)**	Joseph	Challenge Solved via simple guessing of Administrator Password "admin123".	Enforcing strict password policies: special characters, upper and lower case, length, password e.
Login Admin	**Injection**	Joseph	Challenge Solved in tandem with Password Strength (above). Completed again using SQL injection. Using ' OR 1=1-- closes the SQL string, creates a true Boolean statement and converts anything after to a comment. Entering any value into the password to activate the login button. This bypasses the password check and allows any known, registered user account to login.	Using secure APIs to ensure that the data stored in SQL databases is kept separate from and thus not directly modifiable from input fields within the web application. Using server-side rules/sanitisation to prevent use of special characters. Eg: Permitting only '@' symbol for email address fields.
Login Bender	**Injection**	Joseph	Challenge Solved in similar fashion to Login Admin. Bender email address found in review of Banana Juice and exploited SQL injection. OR 1=1 statement here is not successful for the challenge as it logs the Administrator in instead of the Bender account. Challenge completed using: bender@juice-sh.op'--	
Reset Bender's Password	**Broken Authentication (Authentication Failures)**	Joseph	Challenged solved. Sniper Attack via Intruder in Burp Suite to brute force Security Question and change password. Status code for correct payload differs from others (200: successful vs 401: failed authentication).	Replacing Security Question inside the Forgotten Password process with reset links, or one-time passwords sent to known user email address/secondary email address or telephone number. Add rate limiting to prevent attackers from performing brute force attacks.

Expired Coupon	Improper Input Validation	Joseph	Challenge Solved. Scrubbed through main.js to find coupon code information. Researched previous International Women's Day dates and performed manual date change on user device.	Complete removal of archived/expired coupon codes. Ensure that validation is performed on the server side and thus preventing a client-side change (such as the date) from affecting back-end functionality.
Depreciated Interface	**Security Misconfiguration**	Ken	 Searched up b2b and seen available files and xml was there in the code base, so when I uploaded it was allowed even though it said pdf or zips.	• Remove and check deprecated interfaces from the codebase. • Enforce proper access control on all endpoints. • Perform regular audit checks.
Forged Feedback	Broken Access Control	Ken	 Used burp suite to change the ID when I clicked send feedback. It changed to a different user	• Don't trust client side and use server-side trust • id is using secure signing • Ensure feedback is tied to authenticated user
CAPTCHA Bypass	Broken Anti Automation	Ken	 Used burp suite on the repeater to rapidly send reviews as the captcha id was the same and the captcha answer was the same number.	• Have the captcha id hashed and uniquely created • Limit ip sent to sever to three and wait a minute • Use server-side session mapping

Christmas Special	Injection	Ken	 In burp suite change the product id to 10 to add the item was not displayed	• Properly delete the item of the shop instead of hiding it. • Have server side verify what items are allowed or not allowed • Hash the product id so it can't be identified
Forged Coupon	**Cryptographic Issues**	Ken	 It uses z85 encoding so I got the code decoded and figured it is made by month, year and percentage of discount. You can do the same thing, change the number to 80 and get amount	• Use proper cryptographic techniques instead of basic encoding. • String random unique values instead of basic code like may2026-40 • Have server-side authenticate the coupon instead of trusting client with the right code

Vulnerability Analysis

Injection [Joseph]

a. Description & Root Cause

The Injection vulnerability allows attackers to enter malicious inputs into text fields within a web application which "causes the interpreter to execute parts of that input as commands". [JC1]

If user input is not properly validated, filtered, or sanitised before being processed by the system, this can result in the system being abused by attackers who modify database queries to gain access to user accounts (as in the case of the challenges discussed in this section), or retrieve sensitive data from the database, including personal information and credentials.

b. Rationale/Investigative Process

Basic browsing of the Juice Shop interface provided one of the first clues to access the Admin account: the admin@juice-sh.op email address. The Admin had left a review on the first product on the website.



A number of basic passwords were attempted and, surprisingly, "admin123" allowed for successful access to the Admin user account. This was more a Brute Force attack, even if it only took 3 or 4 attempts to gain access.

To attempt a more technical method of access, we can consider that perhaps the user login information is being stored in an SQL database. We can test to see if proper sanitisation has been completed.

By using the Intercept and repeater tools of Burp Suite, a "single quote character and looking for errors", instances of SQL functionality around the user login page are visible. [JC2]



c. How The Exploit Works

We can continue this process by using basic SQL commands to attempt to bypass password requirements.

Using ' OR 1=1-- closes the SQL string, creates a true Boolean statement and converts anything after to a comment. Entering any value into the password to activate the login button.

This bypasses the password check and allows any known, registered user account to login.



The process for logging into the bender@juice-sh.op account is quite similar. However, attempting to use the same ' OR 1=1-- command logged the Administrator in.



In order to successfully log into the desired user account, simply removing the 1=1 Boolean statement from the email field was successful.



d. Potential Impact

The impact can be quite devastating as noticed when completing some of the Juice Shop challenges. Utilizing basic SQL commands allowed for complete access to both the Admin account and the Bender user account.

Assuming an Admin has access to potentially sensitive data and information regarding their users, staff or intellectual property, this data could be held for ransom or sold by black hat hackers. This could include personal information, financial or health records or government secrets.

If these kinds of attacks are successful and such information is attained by attackers, this could cause serious damage to a company's infrastructure, reputation and could result in loss earnings.

e. Recommended Mitigation

By using secure APIs to ensure that the data stored in SQL databases is properly handled and not modifiable by attackers using SQL queries. This prevents input fields from being points of attack.

An additional mitigation technique would be to use server-side rules/sanitisation to prevent use of special characters in fields that could be targets for attackers.

Broken Authentication [Joseph]

a. Description & Root Cause

Authentication Failures are vulnerabilities where attackers "is able to trick a system into recognizing an invalid or incorrect user as legitimate" [JC2] and can take advantage of weaknesses in user identity verification. If proper authentication is not performed on Web Applications, access to the

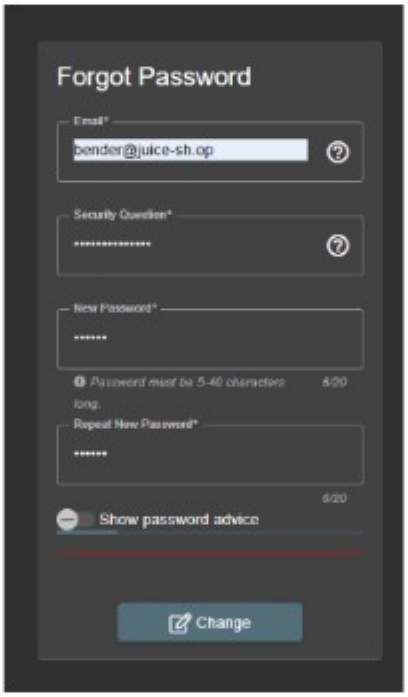
internal functionality as well as private data can be compromised. It is therefore hugely important to ensure that users accessing a Web Application or Service are who they say they are.

Web Applications can be compromised via Authentication Failures such as:

- Brute Force Attacks.
- Weak Passwords.
- Weak/Ineffective credential recovery/Forgotten-Password processes.
- Rationale/Investigative Process

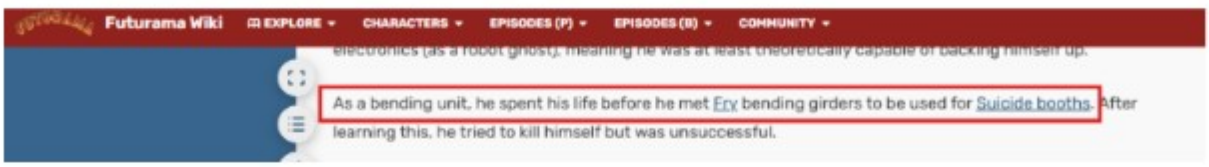
When logging in to the Juice Shop webpage, you see the standard Username and Password fields. Investigating further, a "Forgot Password" link is present.

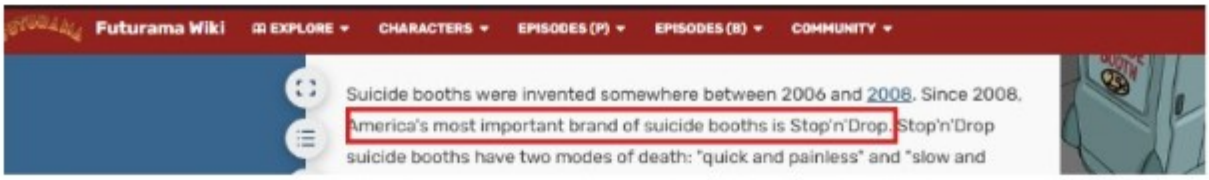
In modern web applications and systems, the Forgotten Password section of a webpage no longer contains static Security Questions, and instead, reset links are sent directly to the user via previously entered contact details, such as email address or telephone number - this is not only to prevent attackers from using public information to answer security questions, but also to try to ensure that only authorized users are the ones who receive their reset password links.



b. How The Exploit Works

The presence of a security question allows for Open-Source Intelligence to be undertaken. Finding the original job for the Bender character from Futurama, "Stop'n'Drop".





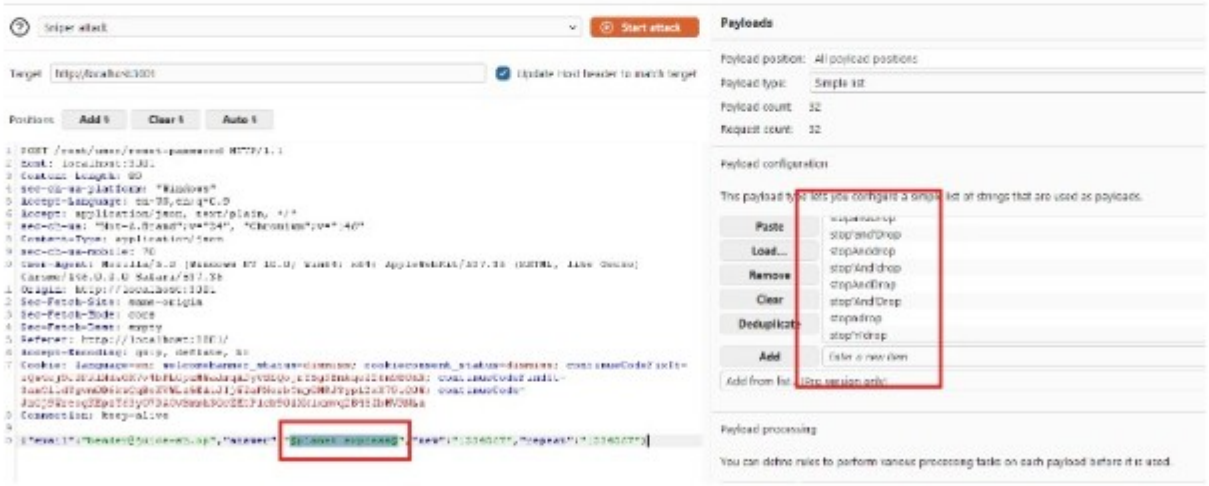
To properly attack this vulnerability, I decided to use the Sniper Attack inside the Intruder section of Burp Suite.

I created a very basic looping python script to generate different versions of the Stop'N'Drop answer, creating variations of lower and upper case letters, and the use of apostrophes.

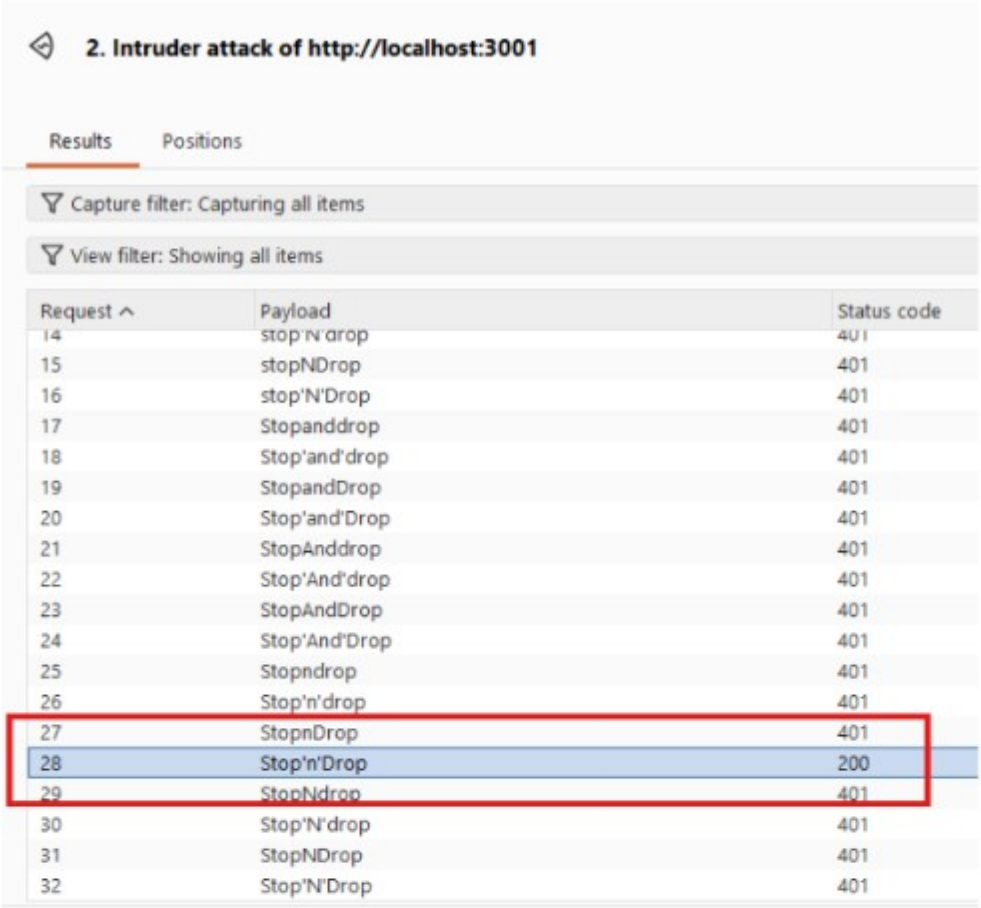




In this challenge, only 32 variations of the password were created. They were entered into the payload section of the Intruder attack, and the Sniper Attack was started. This attack mechanism replaces the originally entered Security Answer with those in payload.



Within a minute or so, the Sniper Attack was complete, and one instance had returned a different Status Code to the rest of the entries.



Using this payload entry as the answer to the Security Question, and completing the other details (Email Address, New Password, Repeat Password), allowed for the password to be successfully changed for the bender@juice-sh.op account.

c. Potential Impact

If Authentication Failures were to happen in a real-world system, the consequences could be quite severe depending on the level of access that the victim account has assigned to them.

Access to emails and data containing sensitive information such as client names, passwords or payment details for example. Should the account have access to health records or governmental information, such information could be held for ransom or sold.

d. Recommended Mitigation

In order to prevent successful authentication based attacks, it would be recommended to ensure that organisations are enforcing proper security policies such as password strength (length, upper/lower case, letters, numbers and special characters) and rotation/expiry, multifactor authentication or the use of password managers to allow users to securely store highly complicated credentials.

Companies should also be keeping up to date with modern security standards. In this example, the use of a Security Question was the method by which a password could be reset, as the answer could be found via OSINT. Instead, implement modern password recovery, performed by sending a reset link to the legitimate user's known email address or phone number.

Segregating administrator accounts to ensure that successful attacks of end users do not necessarily provide a gateway for further attacks to accounts with higher level of access to more secure data or backend functionality. Essentially limiting the impact of a successful attack.

Adding rate limiting to backend functionality would also help in protecting against brute force attacks, preventing an attacker from bombarding a system to gain access.

Sensitive Data Exposure [Mark]

a. Description & Root Cause

Sensitive data exposure occurs when an application unintentionally discloses confidential information, such as credentials, internal files, or personally identifiable information (PII). This usually results from improper data handling, lack of access controls, or insecure storage solutions. In modern web applications, sensitive information should never be exposed to the client unless strictly necessary and handled securely.

This vulnerability can be related to these classifications:

- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor [MD1]
- OWASP Top 10 A02:2021: Cryptographic Failures [MD2]

The root cause is the failure to properly restrict access to sensitive resources and the use of insecure storage mechanisms such as hardcoded credentials in client-accessible files.

b. Rationale/Investigative Process

During testing, application resources were analysed using browser developer tools. This revealed that confidential data was embedded within publicly accessible JavaScript files. Additionally, exploration of application directories and file paths indicated that internal resources could be accessed directly without authentication.

The results show that sensitive data is exposed through both insecure storage (client-side code) and improper access control (direct file access via a URL).

c. How the exploit Works

I have searched for a common indicator “password=” in the Juice Shop website’s public files using the browser's developer tools. I have found hardcoded testing credentials in the main.js JavaScript file (Figure MD1).

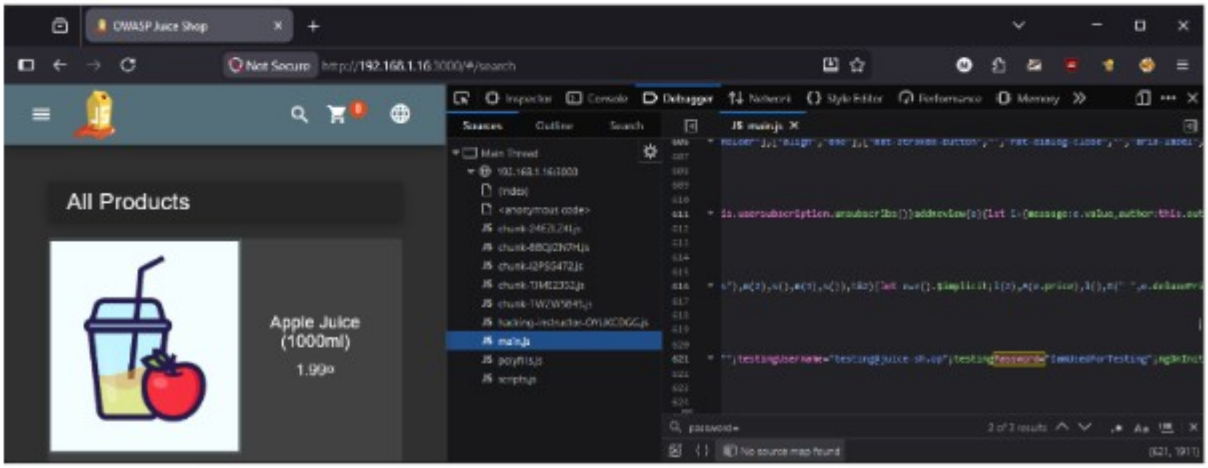


Figure MD1: Hardcoded testing credentials in main.js

As for the exposed 3D model file, here's how I got it:

I thought a 3D-printed logo was the most likely candidate to have a model file.

Based on the challenge hints, I have looked at the product image. I have discovered that the picture was stored at "assets/public/images/products/3d_keychain.jpg" (Figure MD2).

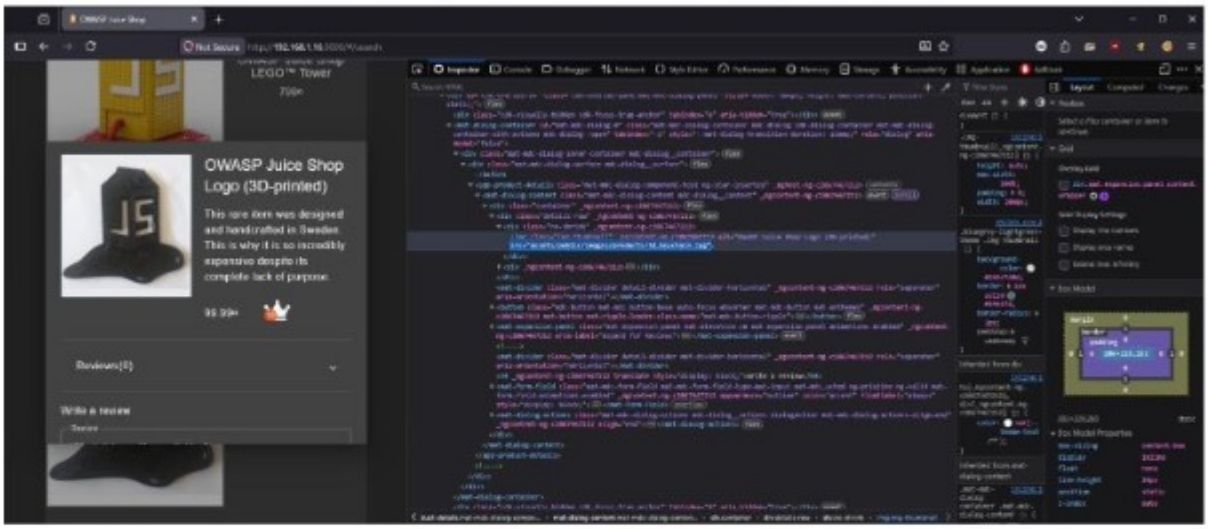


Figure MD2: 3D-printed logo image file reference

From my experience with 3D printing, I knew that an .stl file is the most common format for storing a 3D printable model.

So, I tried different file name variations and finally found that "JuiceShop.stl" is the correct one. I have just downloaded the "blueprint" from the URL "http://192.168.1.16:3000/assets/public/images/products/JuiceShop.stl" (Figure MD3).

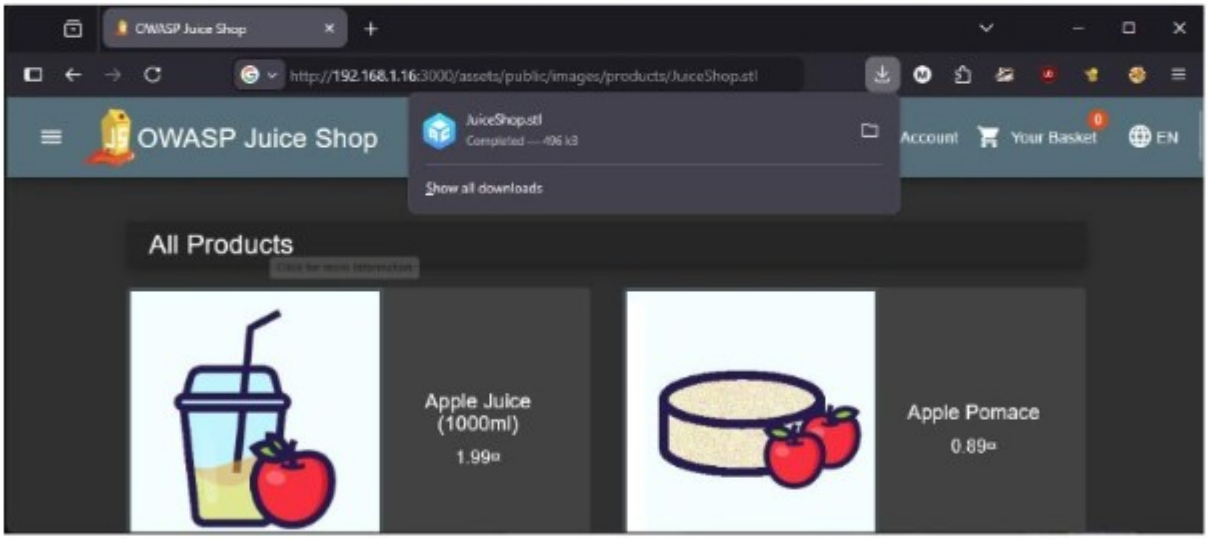


Figure MD3: Accessed 3d model file

d. Potential Impact

The impact of this vulnerability is significant. Exposed credentials may allow attackers to gain unauthorised access to user accounts or administrative functionality. Additionally, access to internal files such as design blueprints may expose sensitive business information.

This can result in:

- Unauthorized system access
- Data breaches
- Loss of intellectual property
- Reputation harm
- Possible regulatory non-compliance (e.g., GDPR)

e. Recommended Mitigation

To mitigate this vulnerability, you should store sensitive information and files in private, encrypted storage that is not accessible to normal users. Remove any hardcoded credentials. Additionally, appropriate access authorisation controls should be implemented for pathways to accessing these files/information.

Improper Input Validation [Mark]

a. Description & Root Cause

Improper input validation occurs when potentially malicious user input is not properly sanitised/validated before it is processed in the application's code. This might cause unexpected application behaviour and violate the desired restrictions.

This vulnerability can be related to these classifications:

- CWE-20: Improper Input Validation [MD3]
- OWASP: Improper Data Validation [MD4]

The root cause is reliance on client-side validation without enforcing equivalent checks on the server.

b. Rationale/Investigative Process

During the investigation of the web application, a complaint form with a size-limited file upload was identified.

To verify that the 100 KB file size limit was properly enforced, a 50 KB test file was uploaded via the form, and the corresponding POST request was captured in Burp Suite.

The POST request was modified to include a raw file string exceeding the 150 KB limit. Which was successfully uploaded to the server, breaking the stated 100KB file size limit (Figure MD4).

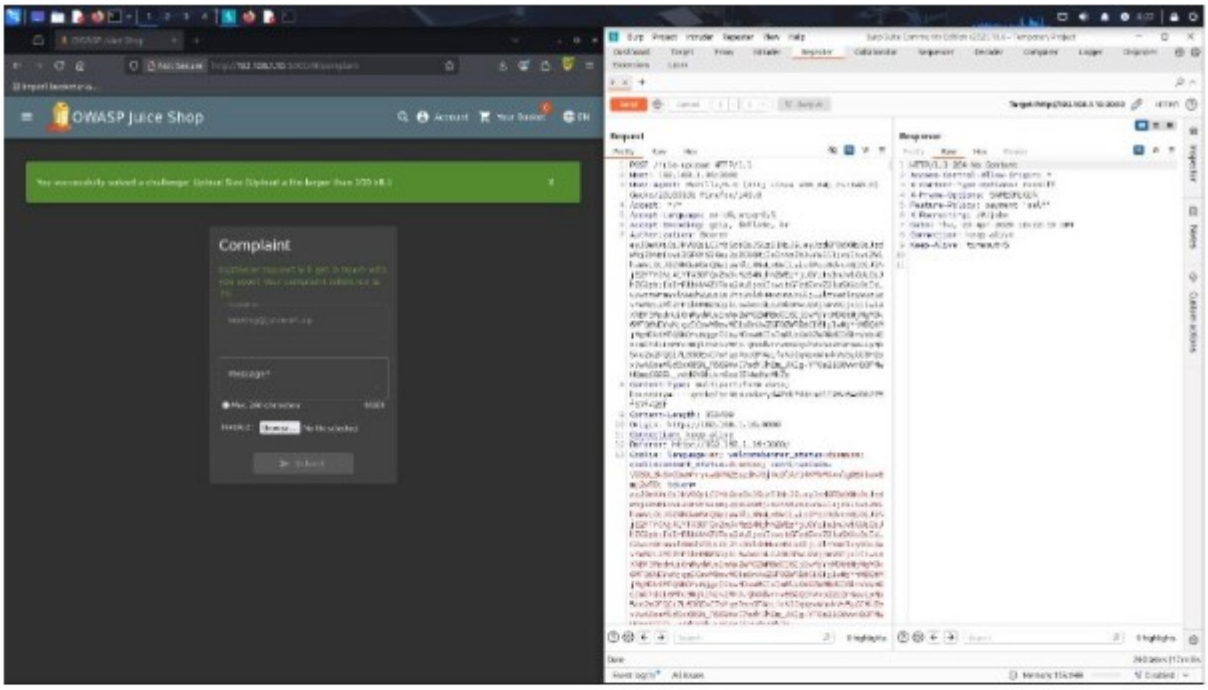


Figure MD4: 150KB file upload

Subsequently, to test out how much larger a file can be uploaded using the aforementioned method. The modified request with a 400KB file string triggered a 500 internal server error with the title "MulterError: File too large," signalling that a secondary verification process with a higher trigger limit is running on the backend server (Figure MD5).

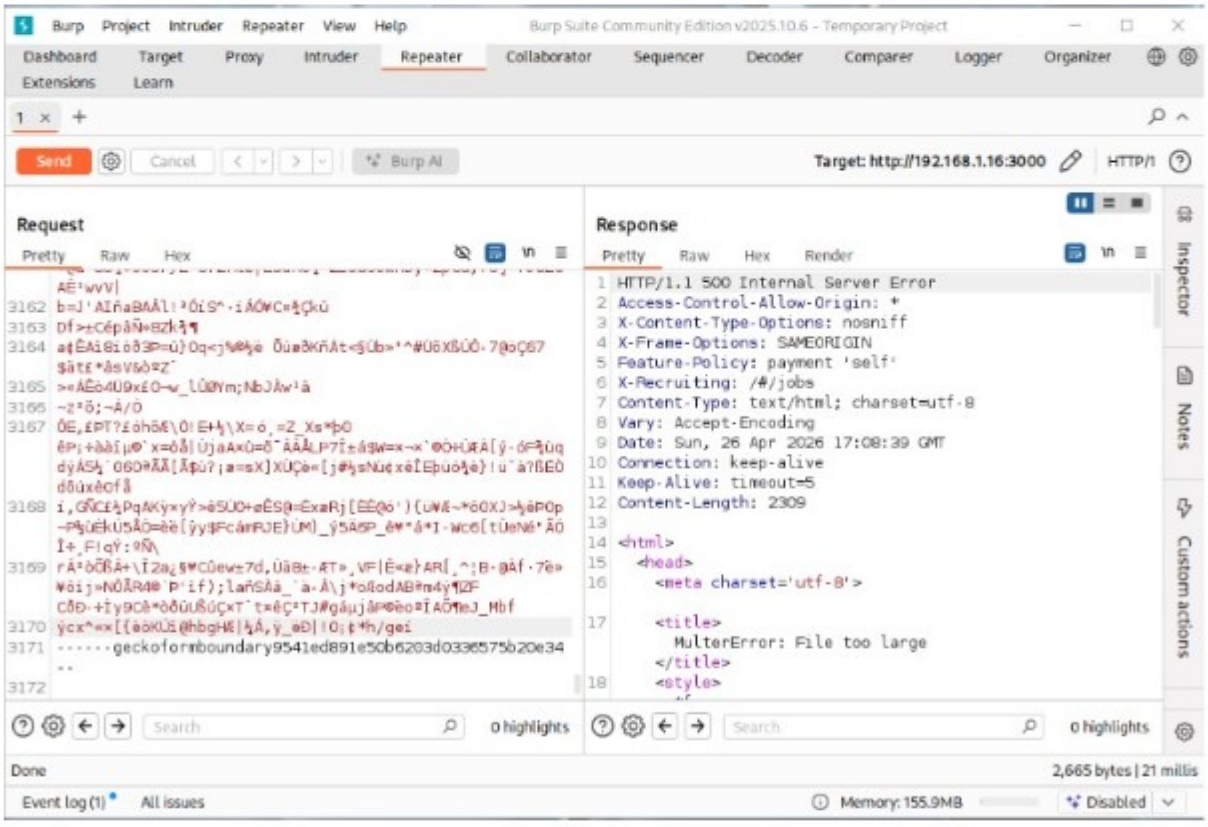


Figure MD5: 400KB file upload

c. How the exploit Works

The exploit works by bypassing client-side validation mechanisms. The application enforces a file-size limit on the frontend. However, this restriction can be bypassed by intercepting the HTTP request with Burp Suite or another similar tool.

By changing the request body to include a file string larger than the limit, you can submit files larger than 100KB to the server. This is because the server backend is not applying the limit to the lower thresholds; the modified request is accepted and processed successfully.

This shows that depending entirely on client-side validation is not enough, because adversaries can manipulate requests before they reach the server.

d. Potential Impact

The impact of improper input validation in this case is moderate, but still significant. The ability to upload files exceeding the stated limit indicates that validation methods are inconsistently applied on the client and server sides. This vulnerability can result in excessive storage, processing power, and internet bandwidth usage.

Even though a second validation process exists on the server side, it was still triggered only for a much larger file upload of 400KB, exceeding the 100 KB limit. This vulnerability can facilitate adversaries in launching denial-of-service attacks by exhausting resources against the web application, leading to revenue loss due to reduced server availability.

Additionally, improper input validation can increase the risk of other attacks, such as the upload of malicious or specially crafted files, depending on how the application handles uploaded content.

e. Recommended Mitigation

To mitigate this vulnerability, strong input validation mechanisms should be implemented and enforced on the server side. File size limits should be clearly defined and applied consistently to block unwanted files before processing.

Additionally, files should be checked for properties beyond size and extension. For example: type, content, and structure. This can reduce the risk of malicious files being uploaded and processed.

In general, client-side checks shouldn't be relied upon. Although they can still be implemented to enhance user experience. Proper logging and monitoring of file uploads should be present to detect abnormal behaviour, such as repeated large file uploads.

Broken Access Control [Maël]

a. Description & Root Cause

'Access control, sometimes called authorisation, is how a web application grants access to content and functions to some users and not others. These checks are performed after authentication and govern what 'authorised' users are allowed to do' [MC1]. Broken Access Control happens when an app fails to impose restrictions on what users are allowed to do. Since authorisation is broken, users can perform actions outside of their intended permissions, as I did in the 'Product Tampering' challenge.

This vulnerability can be related to two security taxonomies, such as:

- Improper Authorization (CWE-285), where the software doesn't perform an authorisation check when a user accesses a resource.

- Authorization Bypass Through User-Controlled Key (CWE-639), where manipulating a product ID allows unauthorised modifications to data.

The root cause is the failure to enforce server-side validation of user permissions for specific HTTP requests. Applications often by mistake rely on client-side controls and security through obscurity instead of a server-side Access Control Matrix.

b. Rationale/Investigative Process

I began by using Burp Suite to intercept the home page GET request to explore my options and noticed that the app uses a RESTful API to load product details (such as descriptions). So, I switched from the public '/rest/' to the internal '/api/' endpoint. I then learnt that using 'PUT' actually updates rather than using 'GET' since it is read-only. And since this app does not check for authorisation, anyone is able to modify page and product details.

c. How the Exploit Works

I first intercepted the GET request from the products page ('/rest/products'), where I could find all currently displayed products (Fig. MC1).

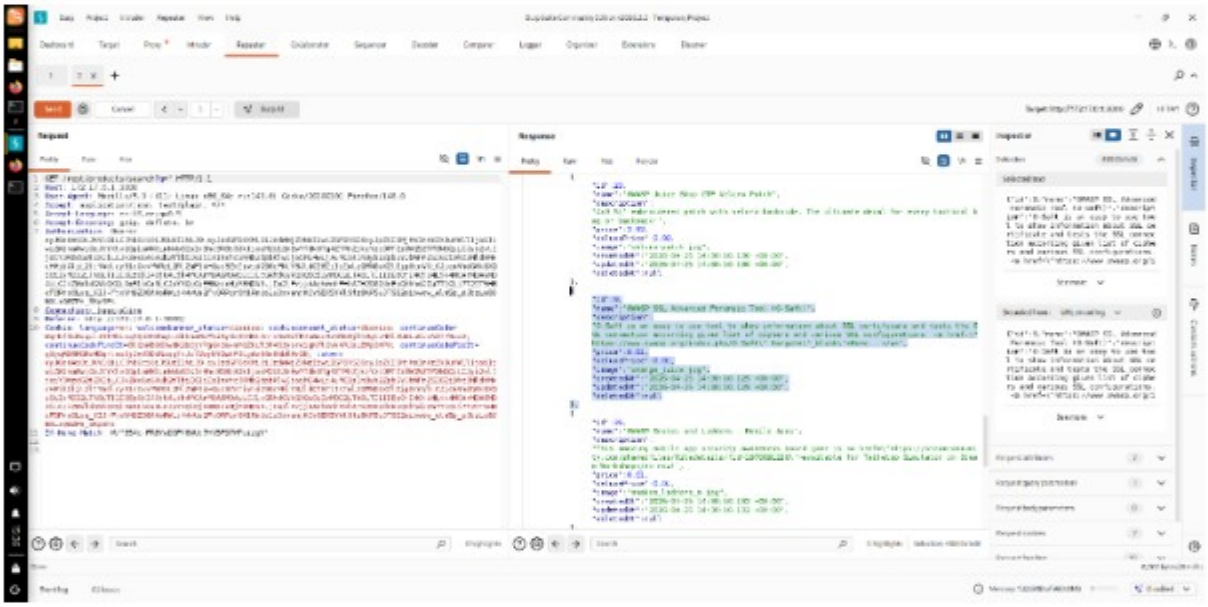


Figure MC1: List of Displayed Products & their Details

I noticed that the products were assigned an ID as well as a description. So, I added a custom description with my own link in the JSON body of the request.

And in the header, I first replaced the 'GET' request with 'PUT' to apply changes, then replaced the endpoint '/rest/products' with 'api/product/9' (to target the specific product of id 9), then copied the response's content-type ('application/json'), and sent the request. And so, with a 200 OK response, when going back to the page, the product's description has changed into my link (Fig. MC2).

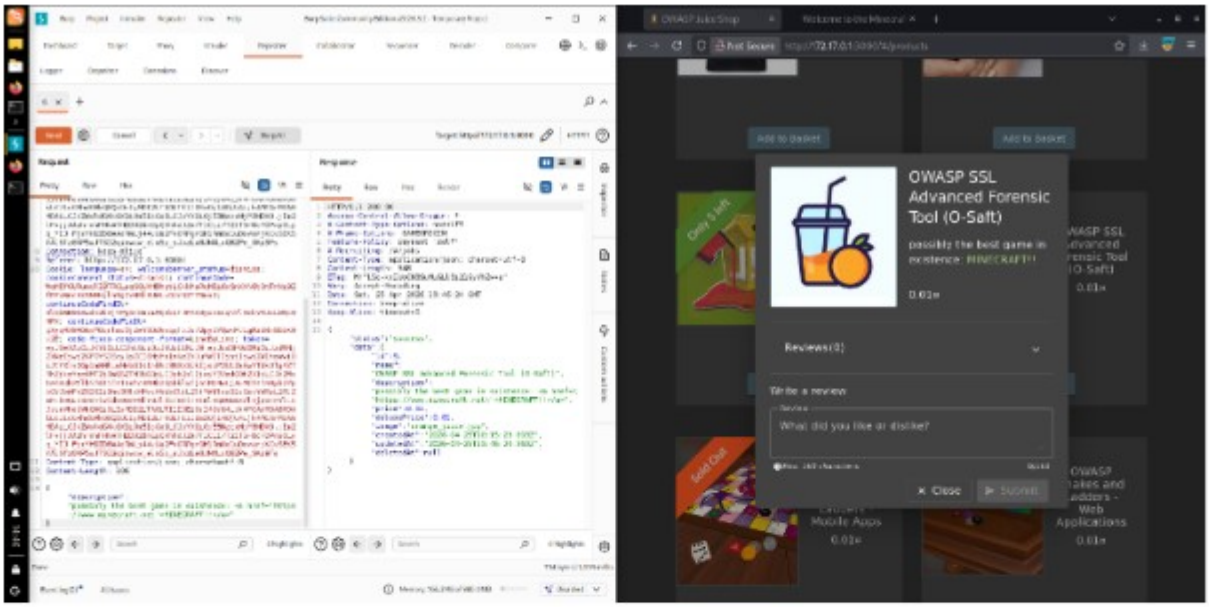


Figure MC2: Modified Product Description

d. Potential Impact

The impact of unauthorised product tampering is quite severe, such as integrity compromise, where attackers can change prices, redirect revenue or cause financial loss.

Another impact is reputational damage, in which replacing product links with malicious URLs breaks customer trust.

There is also data defacement, where unauthorised changes to content can be used for sabotage.

And finally, privilege escalation, where if the API allows modification of user roles through similar logic, it could lead to system takeover.

e. Recommended Mitigation

To mitigate this vulnerability, I propose that the app should have a deny-by-default rule. This means that there is server-side validation, which never trusts the client-side state, where every request to the '/api/' endpoints must verify that the user's session has the right to execute 'PUT', 'POST' or 'DELETE' methods.

Another mitigation is OWASP SVS alignment, where the app implements controls consistent with ASVS V4 to ensure that access control checks are performed centrally instead of being scattered in the code.

The last mitigation I can propose is to disable unused methods, meaning that if a product page only requires reading data, the web server or API gateways should be configured to block any PUT requests from non-admin IPs or roles.

Observability Failures [Maël]

a. Description & Root Cause

Observability Failures happen when a system doesn't have the monitoring to detect or respond to threats. In the context of the challenge, I did ('Misplaced Signature File'), the vulnerability is not only

about the file access but is also the application being unable to flag or alert bypass attempts in the directory.

The root cause for this is both insecure directory listing and insufficient logging of requests. The bypass involves a Poison Null Byte (CWE-626), but the observability failure is in the app's design, which exposes its '/ftp/' directory and fails to create alerts when the user bypasses file type restrictions. So, without proper monitoring, attackers can get the server's file system without any security response.

b. Rationale/Investigative Process

I found out that the '/ftp/' path shows files and folders as it's the server-side directory structure, and used it to find several interesting files, including one 'suspicious_errors.yml'. Upon selecting it I noticed that there was a 403 forbidden error indicating that only '.md' and '.pdf' files were allowed.

So, I attempted a poison null byte injection by adding '%00' after the URL with the '.md' extension after that. It was no success as I received a 400 Bad Request error, which meant a filter was set. I learnt that using '%2500' rather than '%00' is better as it is double-encoded to go past the initial filter. This works as the server decodes once, which turns '%25' into '%', which then results in '%00', which acts as a string terminator. So, I got access to the contents of the unopenable file.

c. How the Exploit Works

To exploit this, I just used the URL. I navigated to the '/ftp/' path, selected the 'suspicious_errors.yml' file (Fig. MC3), and added the double-encoded poison null byte '%2500' with the valid '.md' extension in the URL.

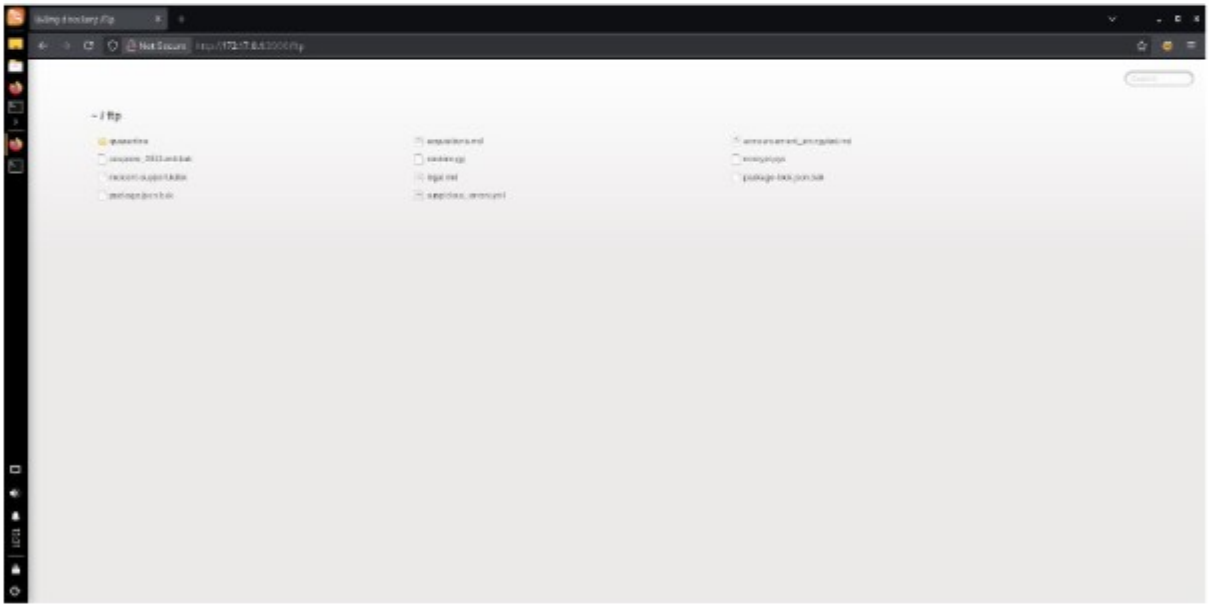


Figure MC3: '/ftp/ ' Path Contents

The file was properly displayed so that I could read its contents (Fig. MC4). And sometimes the file downloads instead of displaying its contents, as 'eastere.gg' did. This worked as the verification layer saw the '.md' at the end of the string, so it allowed the request.

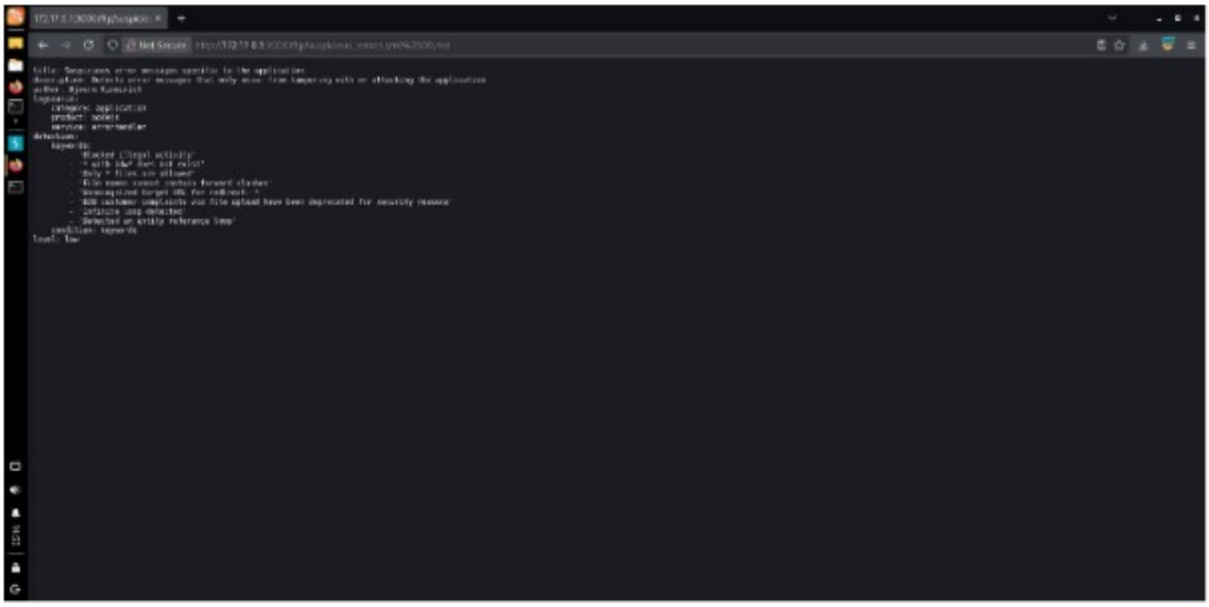


Figure MC4: Contents of 'suspicious_errors.yml' through Poison Null Byte

A similar vulnerability is CVE-158, which shows how improper handling of null bytes allows attackers to access restricted resources.

d. Potential Impact

The main impact of this vulnerability is the fact that the system does not alert on null-byte injections or unauthorised directory exploring, meaning an attacker can stay in for months.

This leads to data exposure, as access to sensitive configuration files (like '.yml') contains API keys or internal logic. It also leads to compliance violations, as failing to log unauthorised access to sensitive directories violates the GDPR framework. And without logs of the bypass attempts, incident responders cannot know who was the first and what the scope of the breach is.

e. Recommended Mitigation

There are several recommendations, such as disabling directory listing to ensure the web server is configured to deny directory browsing.

Another is strict input validation (OWASP C5) as it implements an allow-list that rejects any requests containing null bytes or double-encoding patterns.

Another is enhanced telemetry (ASVS V7) as it configures the app to log and alert 403 forbidden and 400 bad request errors in sensitive paths like '/ftp/'.

And finally, a central alerting system would be useful as multiple failed attempts to access a restricted file from a single IP can trigger a block.

Cryptographic Issues [Ken]

a. Description & Root Cause

Forged Coupon

This challenge makes it possible to reverse-engineer and forge coupons, this means the coupons are predictable or not protected using secure cryptographic procedures, such as hashing or digital signatures. If it were secure, the application would generate coupon codes using strong

cryptographic methods and verify that it is legit on the server side. This indicates attackers can create coupons without needing admin features or access to the server. This is a cryptographic failure because the web app doesn't use secure methods to authenticate the coupon. Without using techniques like signed tokens or implementations of cryptographic algorithms, the web app is vulnerable to forgery

This aligns with:

- CWE-327: Use of a Broken or Risky Cryptographic Algorithm [KS1]
- CWE-345: Insufficient Verification of Data Authenticity [KS2]

The root cause is the lack of cryptographic protection and validation.

b. Rationale/Investigative Process

The first things I'm thinking of to solve this are:

What do the coupon codes look like / are they random or a style or pattern?

Can I access where they have the coupon codes, and if they are encrypted?

Do expired codes work, and if not, why? What errors do I get?

Can I reverse-engineer the codes and make my own?

I went on a fake Twitter site and collected a good few coupons, and I already see a pattern:

k#pDmhz3:t
o*IVjhz3Wr
mNYT0hz3Wr
l}6D#h7ZNq
pes[Ch7ZKp
pEw8ph7ZQr
q:<Irh7ZQr
k#*Agh7ZTs
n(XLuh7ZKp
n(XRwh7ZQr
o*I]qh7ZKp
n<Michz3:t
k#pDmh7ZTs
Ending with :t and Ts is 40%
Ending with Nq or Wr is 20%
Ending with Qr is 30%
Ending with Kp is 10%

This makes me think the last two letters is the percentage of discount

I noticed z3 is 2026 and 7Z is 2025

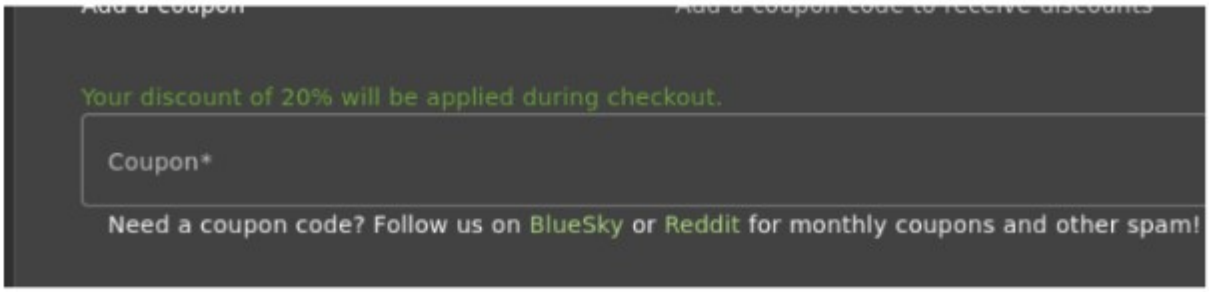
So that would be the month

o*IVjh is march

k#pDmh is April

So, it looks like it is made in the structure month-year-percentage

So, I'm going to test if I can make one using that format.



I'll try april-2026-20%

k#pDmhz3Wr

It did work, and that wasn't code from the fake Twitter. I'm concluding that it is a pattern that I can make up. I just need to figure out how to get higher percentages, so I know how it is encoded. I know it's encoding as it is a pattern, not a random number that spews out random letters. While looking around, I found the ftp directory from the About Us section of the webapp. I saw a file called coupons. I clicked on it and got a 403 error. So, I googled what it is, and you can bypass it through null byte injection. I saw another challenge in the scoreboard called a null byte, so I decided to have a look at how to do it. I tried %00, but it didn't work, so I googled what could be wrong, and I was told that sometimes it needs a double encoding, so %2500 and it works. I got an MD file with coupons.

n<MibgC7sn
mNYS#gC7sn
o*IVigC7sn
k#pDlgC7sn
o*I]pgC7sn
n(XRvgC7sn
n(XLtgC7sn
k#*AfgC7sn
q:<IqgC7sn
pEw8ogC7sn
pes[BgC7sn
l}6D$gC7ss

None of these worked

So, I'm going to check the last letters as this decides the percentage amount

It doesn't work so now I need to know how it is formed so I'll try multiple random letters that will change how it works

I noticed Ty would give 18% and Tz would give 19% and Tx would give 17%

For Wr I changed it to Wy and it gave a 27% and Wz gave 28%

I also noticed for

Ts is 40%

Qr is 30%

Nq is 20%

There are all gaps for 3 the first letter so I can find 80 percent by that logic. But there is a flaw as 50% should be W but it's not as its also 20%

So, my next step would be if there is a file that has the encoding type on it.

I went to the ftp directory and found a file called package.json I went looking for anything that involves encoding these were

"html-entities": "~1.2",

"z85": "~0.0",

"hashids": "~1.1",

"jsonwebtoken": "~8",

"jssha": "~2.3",

"libxmljs": "~0.18"

I used an online encoder to encode z85 to hex then to string and I got APR26-40 and MAR26-20 if I had to guess it's the date and percentage they get off so I'll try encoding it to 80 and see if this works I got k#pDmhz3)x for 80 off and it worked



c. How the exploit Works

Encoding instead of encryption:

- it was using a basic encoding scheme instead of using cryptographic protection. The coupon code was not randomly generated but using the month, year and percentage of discount.
- No server-side validation:
- It doesn't verify if the coupon was officially generated, it only decodes the coupon and applies the discount.

d. Potential Impact

Profit loss

- Since they can manipulate the discount they could be able to add a 99 percent discount and pay way less than the owner wanted to.

Reputation damage:

- If users know that they can exploit coupons they will
- The webapp might be seen as insecure and people will lose trust.

e. Recommended Mitigation

Use of cryptographic signing to secure the coupon code. So, when a coupon is sent the server verifies the signature, so if it is invalid it will reset it.(ASVS 11.3.3) [KS3]

Store coupons on the server-side and generate a random coupon identifier, so when the coupon is entered it checks the database to verify it. (OWASP proactive top 10 C5[KS4]: validate all inputs)

Security Misconfiguration [Ken]

a. Description & Root Cause

Deprecated Interface

For this challenge I will be referencing A02:2025 Security Misconfiguration [KS5] as the deprecated B2B interface wasn't shut down.

This challenge makes it possible to access things that the developer thought they removed. If the application was secure it would properly remove or restrict access. This indicates that an attacker might be able to access restricted or admin privilege using deprecated interfaces that the developer didn't anticipate.

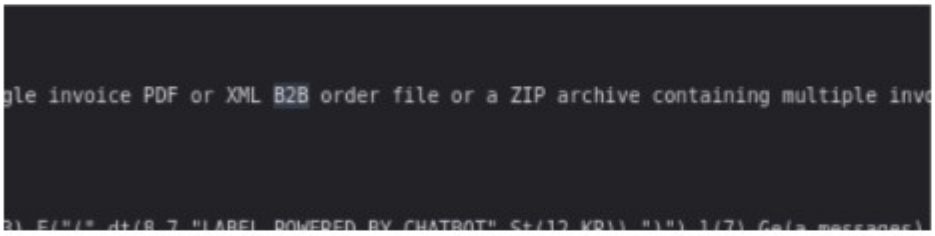
This aligns with:

- CWE-284: Improper Access Control [KS6]

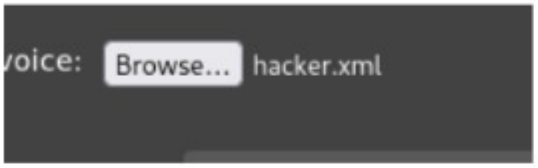
The root cause is that the deprecated interface was not properly removed or secured, this results in missing or insufficient access control checks on interfaces that were supposed to be removed. The developer did not audit or verify the codebase.

b. Rationale/Investigative Process

Since this is checking what wasn't removed I will be checking the main.js file and I will search for b2b



I managed to find it, and it is possible to send a pdf zip or xml file to the system. Looking at the juice shop, you can find the complaint for, and you can send an invoice file so I'm going to make a xml file and send it to see if this works.



I managed to get it working when I hit submit



c. How the exploit Works

The developers forgot to remove the deprecated b2b interface. So, by inspecting the source code on main.js file they were able to see what files are allowed to be sent. The file is sent and processed normally and not verified.

d. Potential Impact

- Unauthorised access to hidden features
  - Attackers can use it to gain access to other features that the developer did not remove and could potentially gain admin or internal features.
- Bypassing security controls:
  - Deprecated interfaces often lack security updates and checks. This allows to bypass the security checks and enable the attacker to inject or read a xml that could be infected or be leaked.

e. Recommended Mitigation

- Remove and check deprecated interfaces from the codebase.
- Enforce proper access control on all endpoints.
- Perform regular audit checks.

SAST Testing

Testing

SAST testing is a method of analysing Web Application and other web scripts which "can help analyse source code or compiled versions of code to help find security flaws". [JC4]

It was decided to perform SAST testing using the Semgrep tool. This is due to the ease in which it could be installed into either Windows or Linux operating systems.
For convenience, running Semgrep within PowerShell on Windows was ultimately the decision that we made as a group.

Instructions to successfully install Semgrep were found directly on their website documentation [JC3] and were very straightforward to complete. These instructions included ensuring the UTF-8 encoding was used within the PowerShell environment in order to prevent errors occurring when scanning the Juice Shop files, applying the specified SAST rules and generating the scan results.

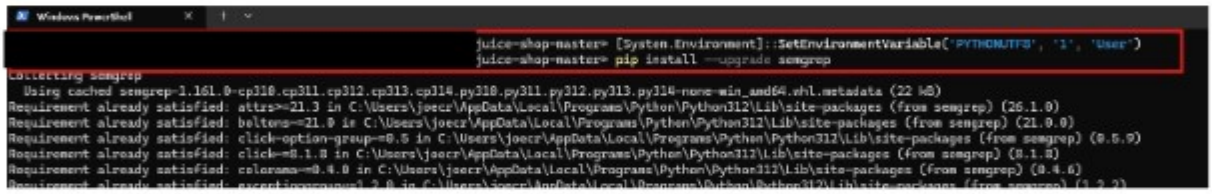


Fig. 1: Semgrep Installation Commands.



Fig. 2: Successful Semgrep Installation.

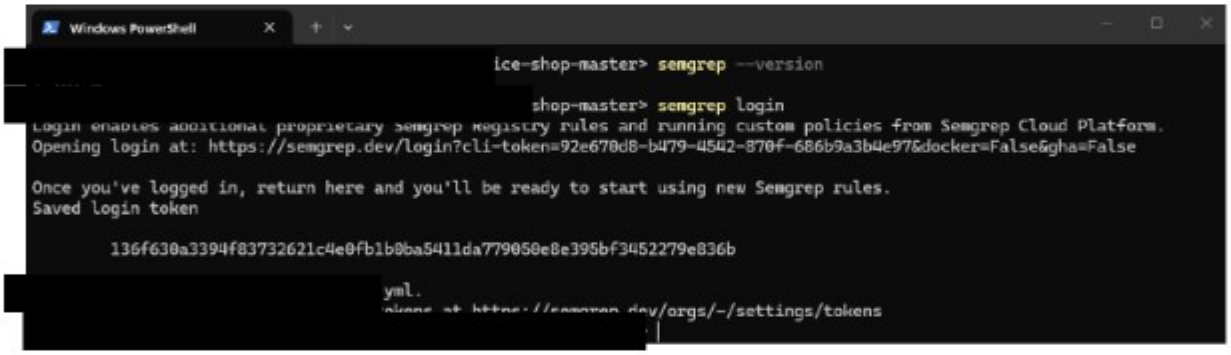


Fig. 3: Successful login to Semgrep

The OWASP Juice Shop source files were downloaded directly from GitHub to ensure a clean, local instance of the web application codes for analysis. It was then decided that the OWASP Top Ten would be used as the code base repository to scan the Juice Shop source files for vulnerabilities and security flaws. [JC 5]

After these tasks were performed, it was possible to implement the Semgrep scan of the Juice Shop directory: semgrep scan --config p/owasp-top-ten --json -o juice-shop-results.json

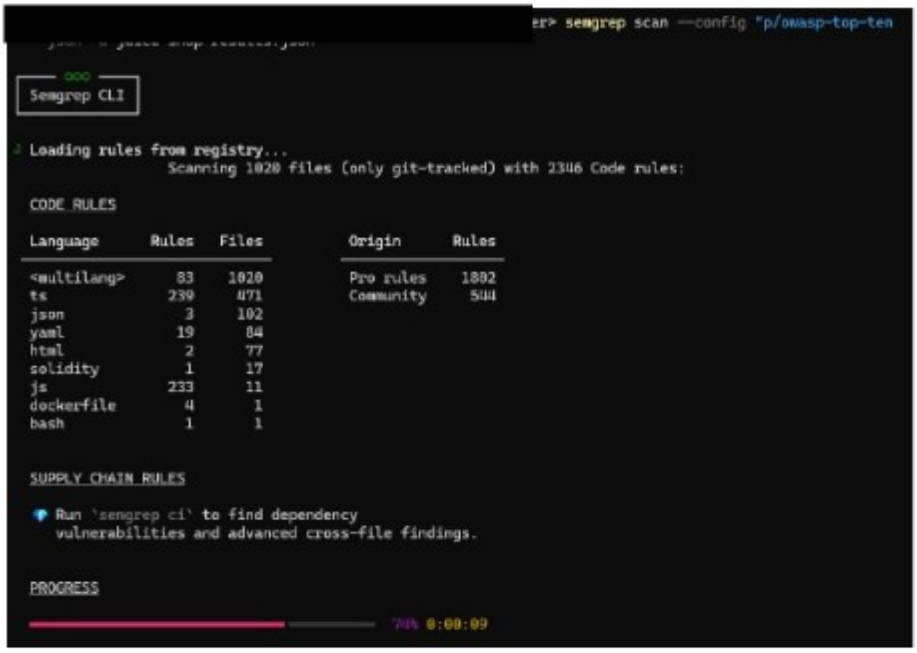


Fig. 4: Semgrep scan in process.

The Findings

The Semgrep scan then complete successfully finding 38 Results which were exported to a JSON file for further analysis.

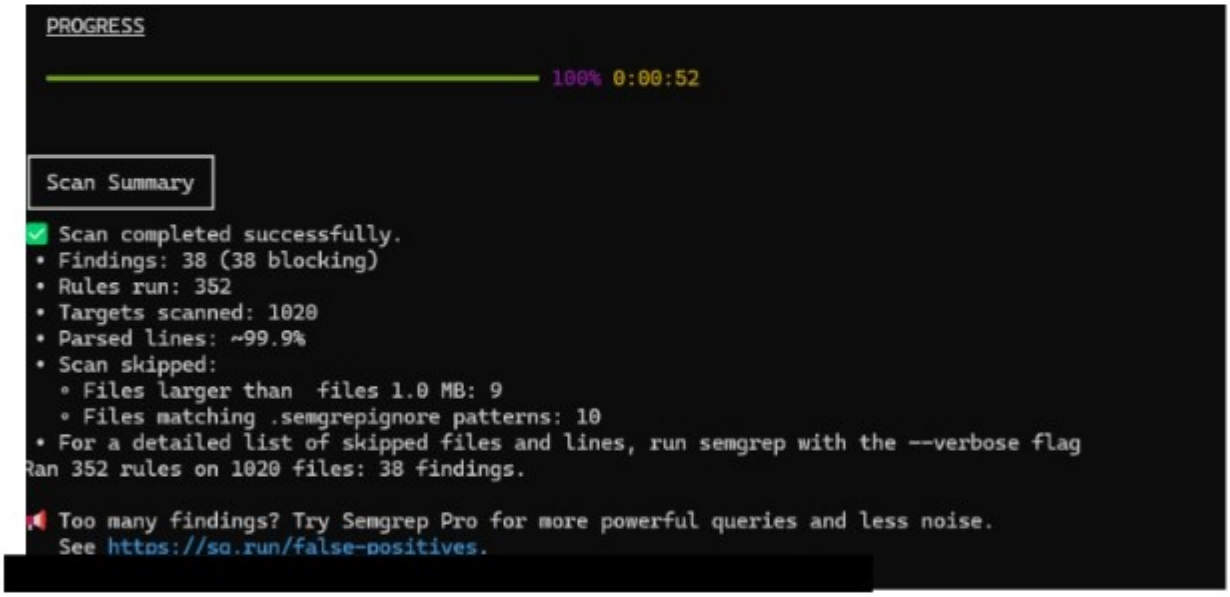


Fig. 5: Complete Semgrep scan

The exported file required formatting to "Pretty Print", as the output was a very large and not very manageable block of JSON data. It was finally possible to begin the STAT analysis of the JSON output.

Analysis

Vulnerability 1: SQL Injection

- File Path: "juice-shop-master\\data\\static\\codefixes\\dbSchemaChallenge_1.ts"
- CWE: CWE-89 Improper Neutralization of Special Elements used in an SQL Command [KS7]

- OWASP Category: A05:2025 Injection [KS8]

Nature

Reading the json file, the database schema caught my eye. The severity was marked as critical and I went to investigate why. The application is taking user input and putting it directly into a database query without checking it. This means an attacker can use SQL injection



The main risks of this are:

- Using ' OR '1'='1 to bypass authentication
- Using UNION attacks to get data like passwords or banking information
- Changing or deleting data in the database

App behaviour

I was checking the dataflow trace, and it said it was a tainted source. This is where intrusted or external an application may be produced

Where they source the data is from 'req' which is the http request the user sends from their browser, this means the user is in control of the data

The validation state really caught my eye as there isn't any at all. This means it's not checking input , no filter and sanitisation

This means they can:

- Inject their own SQL statements
- Bypass login
- Access data they shouldn't have access to

Impact and Analysis

The severity being critical makes sense to me because SQL injection is a very serious vulnerability. The likelihood is high since there is no validation at all.

The confidence is also high because I saw the dataflow trace that the input goes from 'req' straight into the query.

Overall, the security impact is very high because it could lead to full access to the database, including sensitive data being exposed, changed, or deleted.

Recommendations

- Use parameterized queries
- Validate and sanitise all user input
- Don't directly insert user input into SQL queries
- Limit database permissions where possible

Vulnerability 2: Server-Side Code Injection

- File Path: 'juice-shop-master\routes\userProfile.ts'
- CWE: 'CWE-94: Improper Control of Generation of Code ('Code Injection')' [MC3]
- OWASP Category: 'A03:2021 – Injection' [MC4]

Nature

The scan found a 'CRITICAL' severity issue which has to do with the use of the 'eval()' function, which in JavaScript is a function that evaluates a string as code and executes it (Fig. S1).

The tool did a taint analysis (it's in the 'dataflow_trace' part of the JSON), it tracks how data moves from a 'Source', which is the request object, containing untrusted data given by the user, to a 'Sink', which is the eval function to call line 62.:

Because the app takes input from the user profile settings and puts it in eval() without good sanitization, an attacker can go past simple text and inject malicious JavaScript commands that the server will execute.



Figure S1: Part of the Server-Side Code Injection Vulnerability

Correlation with Application Behaviour

In Juice Shop, this vulnerability has to do with the User Profile page. So, when a user updates their profile, the app tries to process the username or other profile data.

An attacker can exploit this by entering a 'username' that contains a malicious payload, such as ''); require('child_process').exec('whoami') //'.

When the server processes this, the eval() function executes the exec command, which would let the attacker run operating system commands (it's remote code execution). This could be used to steal the database or shut down the server and more.

Impact and Priority Analysis

This finding is considered 'CRITICAL' and should be a high priority to fix.

Considering everything, the confidence is high, as the data's trace shows untrusted input reaching a dangerous sink (which in this case is eval).

The likelihood is also high as the profile is a normal feature that is easily accessible to any user.

And as for the impact, it is also high as remote code execution is a very severe type of web vulnerability.

Remediation Recommendation

- Firstly, remove eval() and replace it with a safer alternative.
- Have some form of input validation. So, implement an allow-list for usernames, preventing the use of special characters.
- And finally disable functions which are vulnerable to prevent any future use of eval() in the code.

Vulnerability 3: Hardcoded Private Key

- File Path: 'juice-shop-master\lib\insecurity.ts'
- CWE: 'CWE-798: Use of Hard-coded Credentials' [MC5]
- OWASP Category: 'A07:2021 – Identification and Authentication Failures' [MC6]

Nature

The scan found an RSA Private Key hardcoded into a string variable inside of the 'insecurity.ts' library (Fig. S2). Since private keys are used to prove identity or decrypt sensitive data, by embedding this key in the source code, the application violates the principle of Secrets Management.



Figure S2: Part of Hardcoded Private Key Vulnerability

The main risks of this are:

- Exposure, which is permanent, where once a secret is added to a version control system (like GitHub), it stays in the commit history even if deleted in a later version.
- Unauthorized decryption, where an actor who gets the source code can use this key to decrypt traffic or sensitive data meant only for the server.
- And identity impersonation, where if the key is used to sign digital certificates or tokens, an attacker can create their own credentials.

App Behaviour

In the context of Juice Shop, this specific file 'insecurity.ts' is usually used to handle intentionally insecure cryptographic functions.

To be more precise, this RSA key is used by the application to sign and verify JSON Web Tokens (JWTs). And because the private key is known, a user can do several things, such as:

- Intercept their own session JWT.
- Use the hardcoded key to re-sign a modified token where their role is changed from 'customer' to 'admin'.
- Gain full administrative access to the application because the server uses this same hardcoded key to 'verify' the forged token.

Impact and Analysis

The Semgrep output says that the likelihood and impact as 'MEDIUM', but in a real-world scenario this would probably be escalated to 'HIGH' or 'CRITICAL'.

In terms of confidence, it is high as it is not a false positive. The string explicitly follows the '---BEGIN RSA PRIVATE KEY---' format.

In terms of the complexity inn fixing this, it is low, as the key should be moved to an environment variable or a dedicated server.

And finally for the security impact, it can be considered high as a compromise of this key would lead to a breakdown of the app's authentication integrity.

Recommendations

There are several recommendations to resolve this finding:

- Firstly, revoke the Key. Assume the current key is compromised and generate a new one.
- Then, use environment variables to store the key in a '.env' file or a system environment variable.
- And finally, use a Secrets Vault. Use tools like AWS Secrets Manager or Azure Key Vault to only put the key into the app at runtime without it appearing in the code.

DAST Testing

Testing

DAST is important in application security. It's different from SAST, which analyses source code. DAST interacts with a running application, simulating the behaviour of a malicious actor.

The goal is to identify vulnerabilities, misconfigurations and security flaws, with manual validation to eliminate false positives, and show impact. The tool used for this was ZAP (Fig. D1).

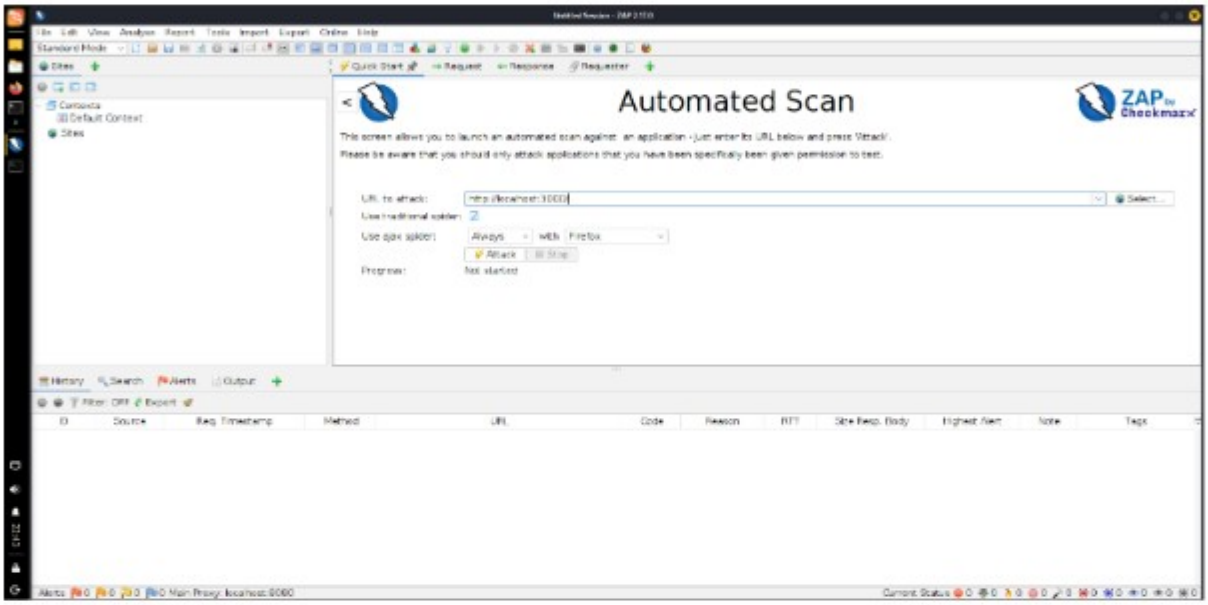


Figure D1: Scan Setup

To ensure getting the most results, both spider methods were used. The traditional spider, which was used first to map the static structure of the application, identifying directories, API endpoints (such as the '/rest' and '/api' directories), and static files (such as '/robots.txt' and '/sitemap.xml'). And the AJAX spider, which was then used on Firefox. This spider interacts with the Application Object Model and the Document Object Model, meaning it clicks buttons, fills out forms, and executes JavaScript to find dynamic routes.

To prevent long automation loops, these constraints were used:

- The target URL 'http://localhost:3000' was the default context. All third-party domains were excluded from the attack.
- The AJAX spider was constrained by maximum crawl depth limits to prevent infinite recursive crawling.
- Instead of executing a full scan against all endpoints, the active scan was targeted. As seen in the site tree (Fig. D2), specific high-value directories such as '/rest/' (which handles API logic) and '/ftp/' (which handles file storage) were prioritized.

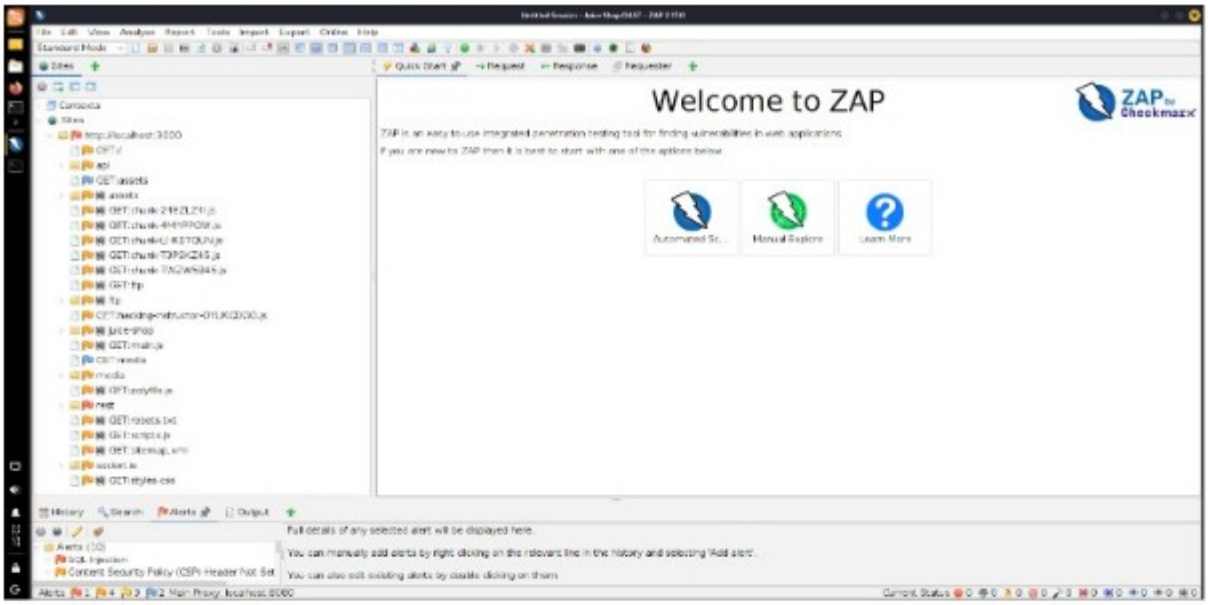


Figure D2: Completed Scan / Site Tree

The Findings

After completing the active and passive scanning, ZAP generated alerts sorted by severity (Fig. D3). The alerts tab showed a total of 10 different vulnerability categories, showing both targeted input validation failures and server misconfigurations.

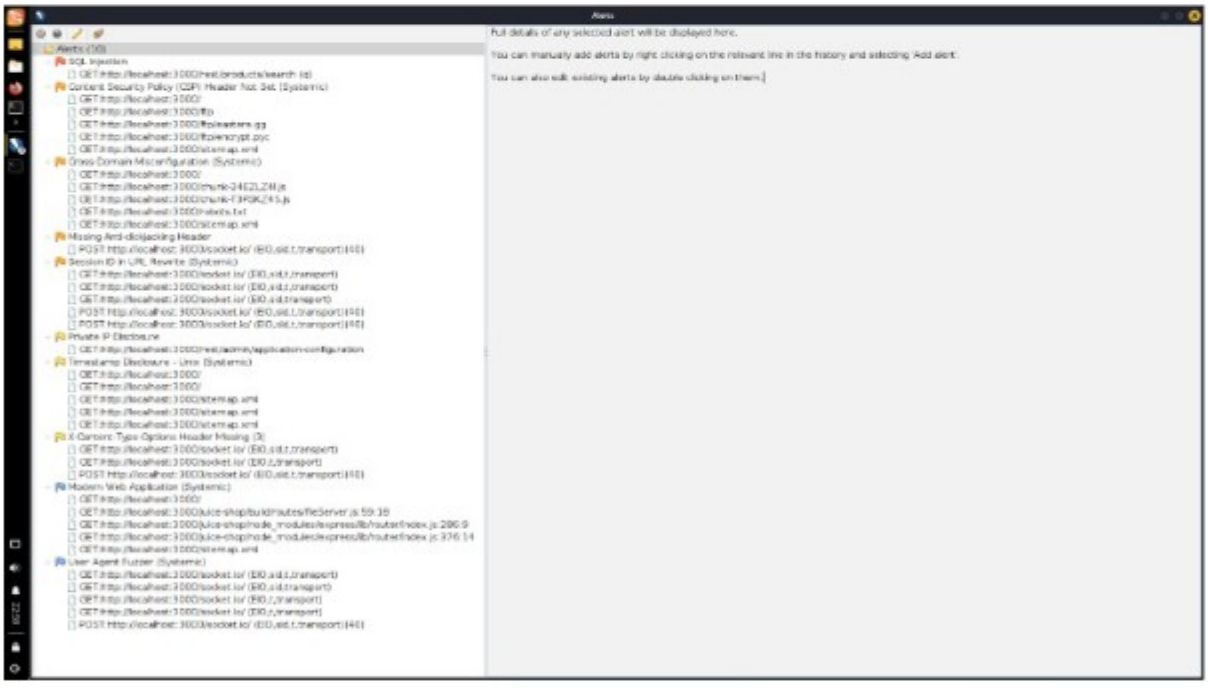


Figure D3: DAST Alerts

As seen in the picture, these are the following alerts:

- High Severity
    - SQL Injection (GET /rest/products/search)
- Medium Severity
    - Content Security Policy (CSP) Header Not Set (Systemic)

    - Cross-Domain Misconfiguration (Systemic)
    - Missing Anti-clickjacking Header (POST /socket.io/)
    - Session ID in URL Rewrite (Systemic)
- Low Severity
    - Private IP Disclosure (GET /rest/admin/application-configuration)
    - Timestamp Disclosure – Unix (Systemic)
    - X-Content-Type-Options Header Missing
- Informational
    - Modern Web Application (Systemic)
    - User Agent Fuzzer (Systemic)

The 'Systemic' tag shows that some vulnerabilities exist globally across the application's responses instead of being isolated to one page.

Analysis

DAST tools can have false positives, meaning they flag anomalies that don't pose a risk. So, manual interpretation and validation are needed.

SQL Injection (High Severity)

The ZAP flagged a SQL Injection vulnerability on the GET /rest/products/search endpoint. Specifically, the 'q' parameter (which handles user search queries). As shown in the alert details (Fig. D4), the tool used the attack payload '%27%28'.

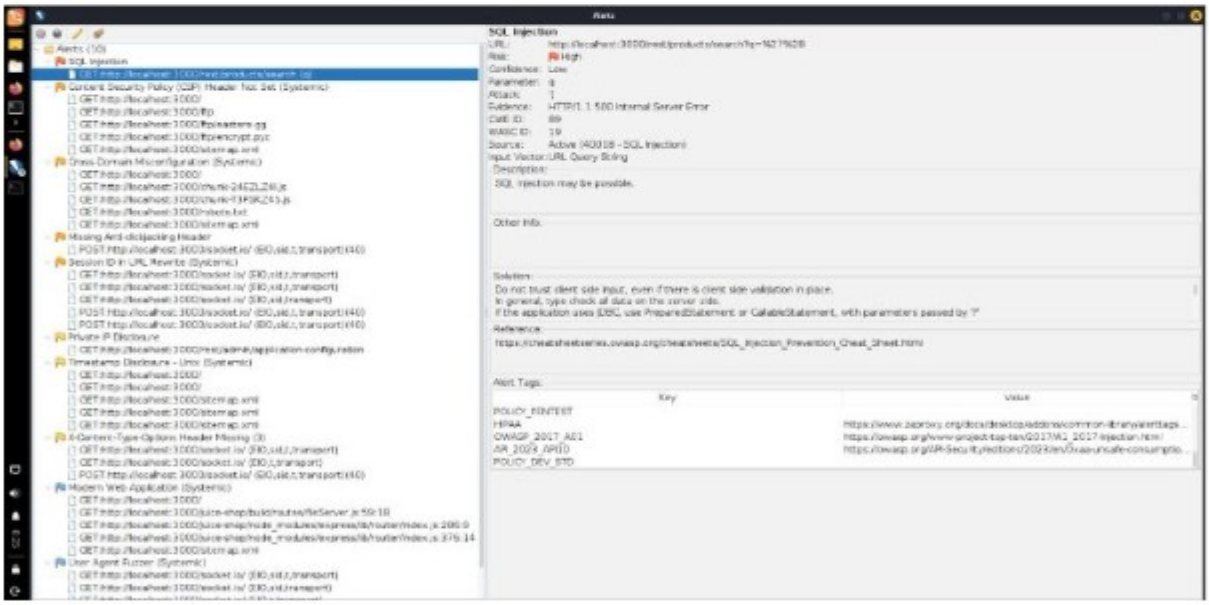


Figure D4: SQL Injection Alert Details

The evidence for this alert was a 'HTTP/1.1 500 Internal Server Error'. A 500 error here suggests that the injected character broke the SQL query syntax, causing the database driver to show an unhandled exception instead of sanitizing the input.

ZAP assigned a low confidence rating to this finding. This means that even if the server crashed (500 error), the tool could not automatically confirm if data could be taken or if the query could be fully manipulated.

To verify if the low confidence crash was a true exploitable vulnerability, we did these steps:

1. We submitted a normal request: 'http://localhost:3000/rest/products/search?q=apple' and the server returned a 200 OK.
2. We then injected the ZAP payload '?q='(' and the server returned a 500 Internal Server Error (Fig. D5), which confirmed that the backends was unable to handle metacharacters.
3. To go beyond a simple crash, we tried to fix the broken query and force a specific outcome using the payload '?q=apple'))—', and the server returned a 200 OK but with zero results, indicating the query was correct but restricted.
4. We then injected a payload made to return all records '?q=apple')) OR 1=1—', in which the app responded with a 200 OK and a JSON response containing every product in the database (Fig. D6).

So, the manual validation was a success and proved that it was indeed a high-risk vulnerability. By seeing the app's change from a 500 error to a 200 OK, we found that the search functionality is directly vulnerable to SQL injection. This confirms that the application is failing to follow 'OWASP A01:2017' (Injection standard) [MC7].

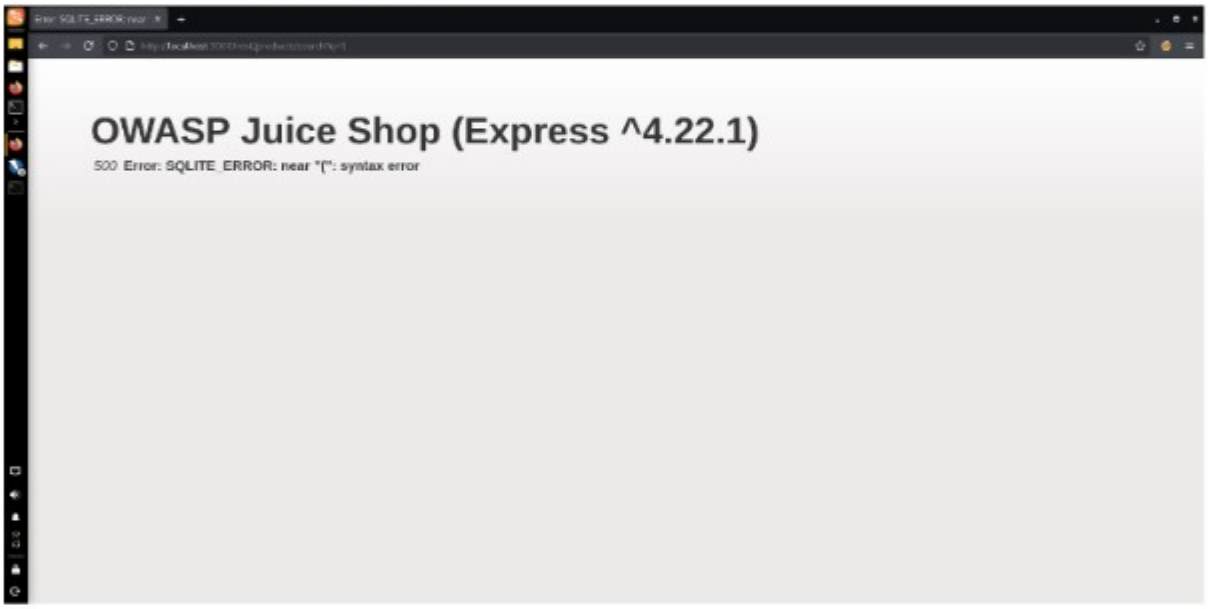


Figure D5: SQL Injection Error

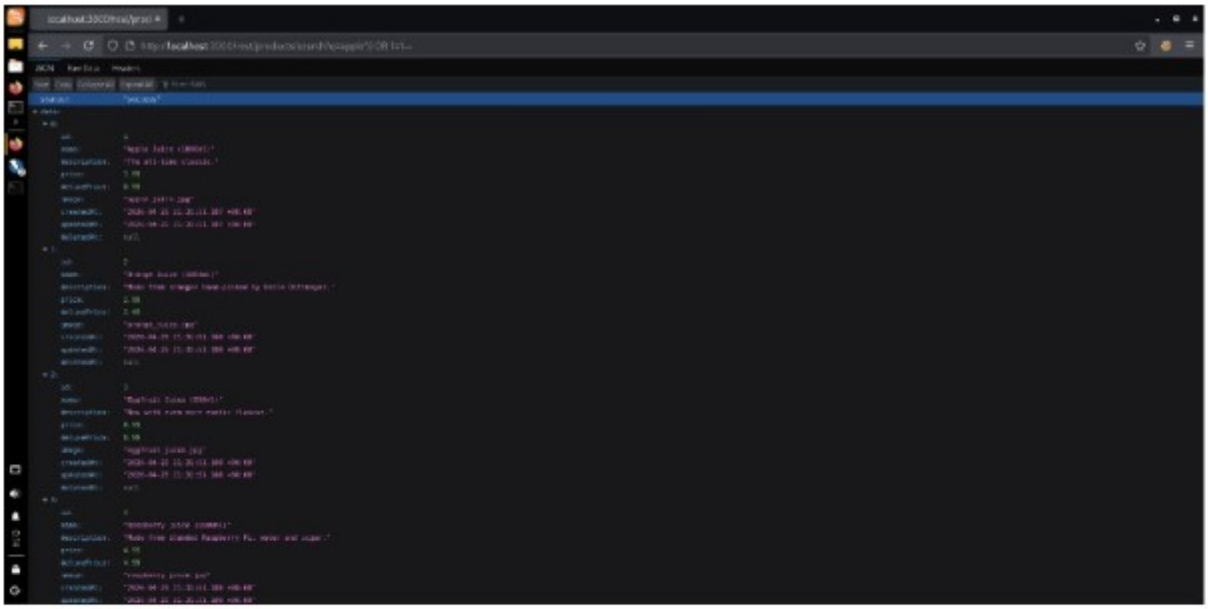


Figure D6: SQL Injection Success

Content Security Policy (CSP) Header Not Set (Systemic - Medium Severity)

ZAP flagged a systemic lack of Content Security Policy (CSP) headers across the application (CWE-693) (Fig. D7). The scanner assigned this a high confidence rating, as the absence of the header in the HTTP response is an easily verifiable fact.

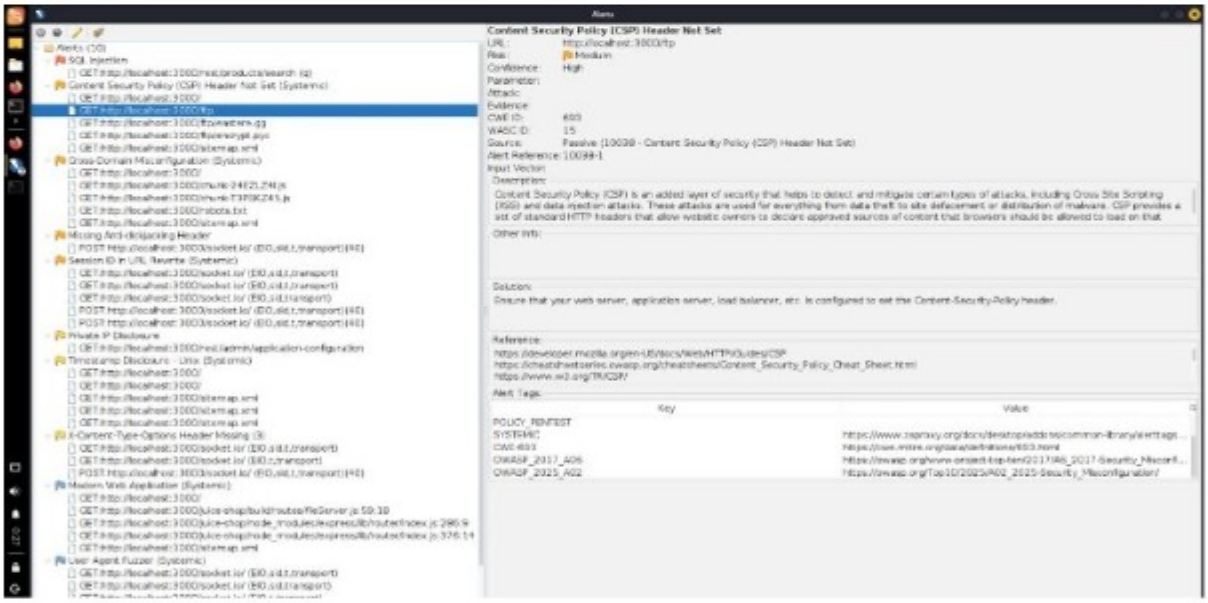


Figure D7: Content Security Policy Header Not Set Alert Details

As [MC8] states, Content Security Policy is an important defence-in-depth HTTP header that contains instructions to restrict resources that a browser is allowed to load (such as JavaScript, CSS, and images). Because Juice Shop is angular and a single page app, the client relies on JavaScript. Without a CSP, the app relies on input sanitization to prevent Cross-Site Scripting (XSS). If an attacker injects a malicious script, the browser will execute it or get external payloads because there is no policy to block non-approved origins.

Cross-Domain Misconfiguration (Systemic - Medium Severity)

The scanner found a Cross-Origin Resource Sharing (CORS) misconfiguration (CWE-264) with a medium confidence rating (Fig. D8). The evidence was the presence of the Access-Control-Allow-Origin '* HTTP' response header.

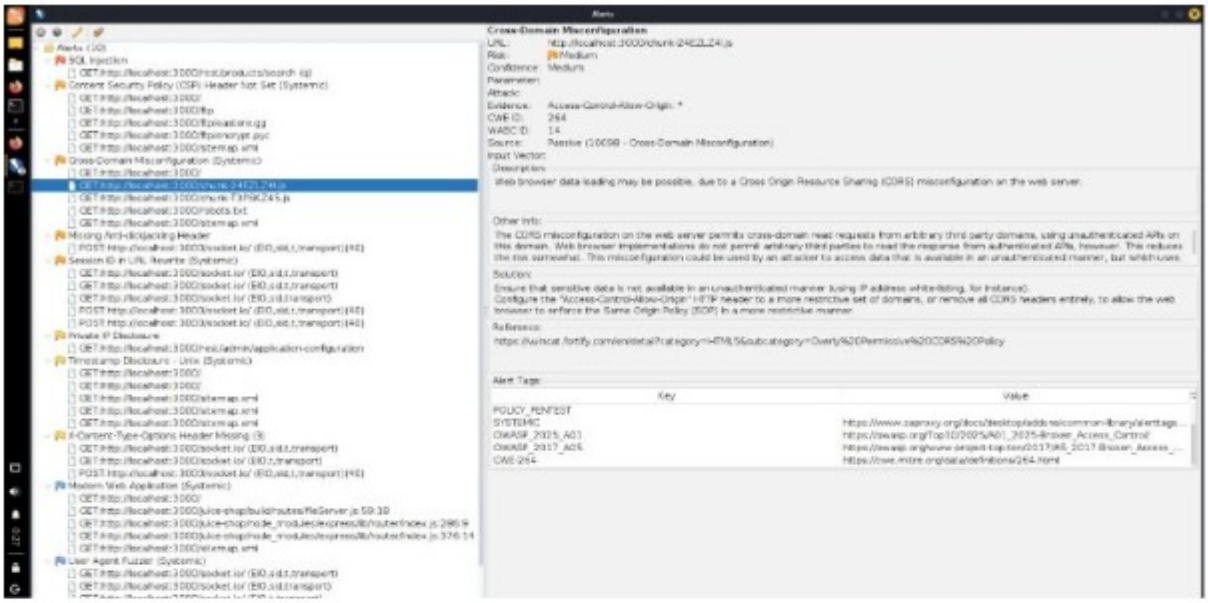


Figure D8: Cross-Domain Misconfiguration Alert Details

As [MC9] explains, Same-Origin Policy prevents a malicious website from reading data from another site. CORS is used to relax this policy. However, returning a wildcard ('*') for the 'Access-Control-Allow-Origin' header permits any third-party domain to make read requests from other origins. This configuration unnecessarily expands the attack surface and can expose unauthenticated data or assets to malicious third-party origins.

Missing Anti-clickjacking Header (Medium Severity)

ZAP found the absence of anti-clickjacking protections (CWE-1021) on several endpoints, such as POST '/socket.io/' (Fig. D9). The scanner found that neither the 'X-Frame-Options' header nor the 'Content-Security-Policy: frame-ancestors' were present in the server's response.

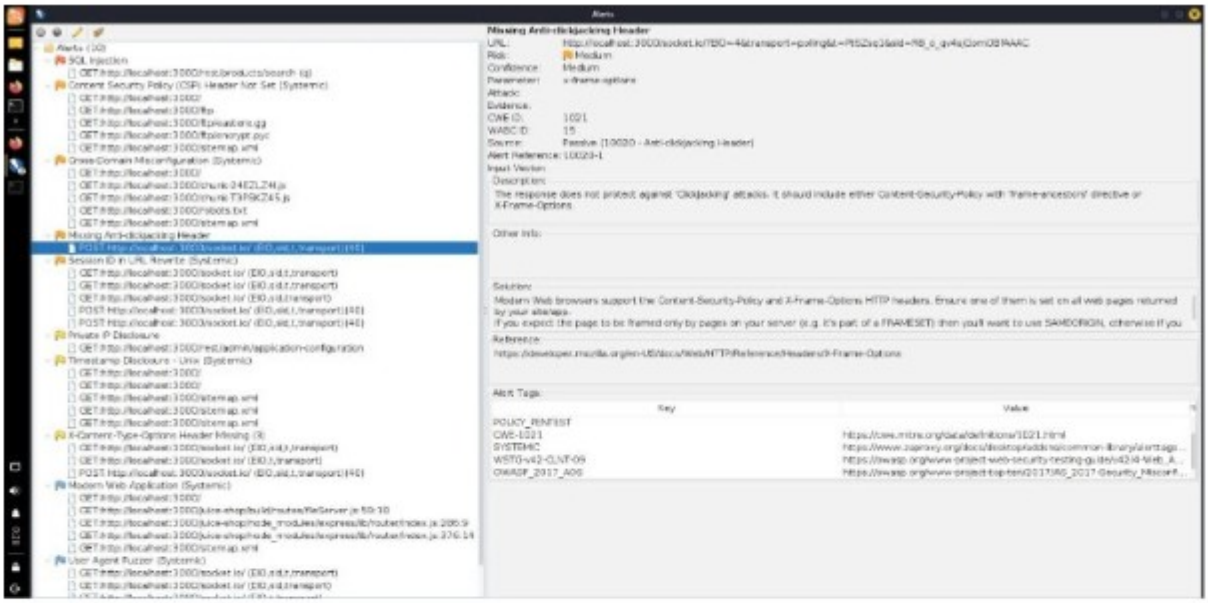


Figure D9: Missing Anti-clickjacking Header Alert Details

[MC10] says that clickjacking happens when an attacker uses frames (like '<iframe>') to trick a user into clicking on action-making elements of the app while they believe they are interacting with the attacker's normal-looking site. By removing the 'X-Frame-Options' header (which should be set to DENY or SAMEORIGIN), the Node.js server permits any external domain to the interface. This could allow an attacker to hijack user clicks to perform unintended actions.

Session ID in URL Rewrite (Systemic - Medium Severity)

The scanner flagged a session management vulnerability (CWE-598) with high confidence (Fig. D10). ZAP got evidence of session tokens being transmitted in the URL query string. More specifically the 'socket.io' endpoint was observed using the parameter 'sid' containing a session id.

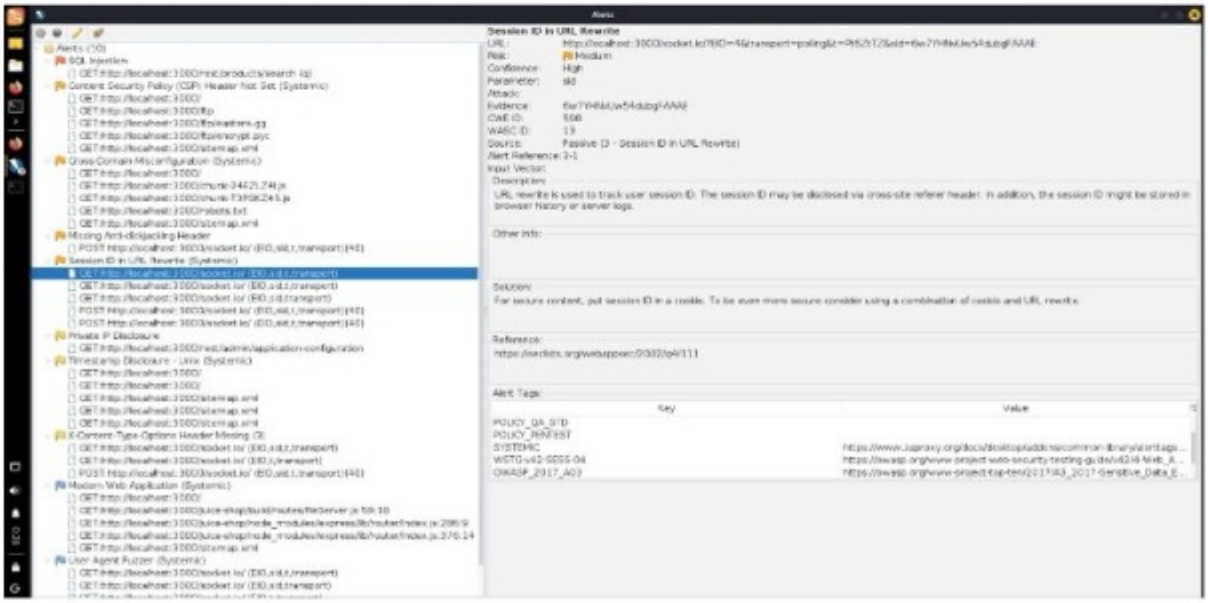


Figure D10: Session ID in URL Rewrite Alert Details

Passing session identifiers or sensitive tokens in the URL is not secure. URLs are exposed in several locations, such as browser history, bookmark lists, proxy or firewall logs, and the HTTP Referrer

header. If a user navigating the app clicks an external link while their session ID is in the URL, that sid will be transmitted to the third-party server though the Referrer header. An attacker with access to any of these could hijack the session without needing to interact with the user.

Remediation Recommendations

Based on the DAST findings, the following recommendations should be used:

- To resolve the SQL Injection vulnerability, the backend must abandon dynamic SQL string concatenation. All database queries should use parameterised queries or Prepared Statements. Basically, ensure that user input is passed safely as set parameters rather than injected directly into query clauses.
- The absence of security headers can be resolved by integrating the Helmet.js library which will enforce secure defaults, such as 'X-Frame-Options: SAMEORIGIN' to prevent clickjacking.
- Remove the wildcard ('*') from the 'Access-Control-Allow-Origin' header.
- Session identifiers should never be transmitted via URL query parameters.

DAST Conclusion

Overall, the scan showed that ZAP is great for finding the basics, and that manual testing matters. we were able to turn a low confidence error to a database leak, which proves that the app does indeed have big security issues. This section of the project showed that a targeted scan is a great way to get clear results without wasting time or getting lost in too much data.

References

Vulnerabilities

[MD1] Nick Johnston. "CWE-200: Exposure of Sensitive Information to an Unauthorized Actor." *cwe.mitre.org* https://cwe.mitre.org/data/definitions/200.html [accessed Apr. 25, 2026]

[MD2] OWASP. "A02:2021 – Cryptographic Failures." *owasp.org* https://owasp.org/Top10/2021/A02_2021-Cryptographic_Failures/ [accessed Apr.25, 2026]

[MD3] Abhi Balakrishnan. "CWE-20: Improper Input Validation." *cwe.mitre.org* https://cwe.mitre.org/data/definitions/20.html [accessed Apr.26, 2026]

[MD4] OWASP. "Improper Data Validation" *owasp.org* https://owasp.org/www-community/vulnerabilities/Improper_Data_Validation [accessed Apr.26, 2026]

[MC1] OWASP. "Broken Access Control." owasp.org. https://owasp.org/www-community/Broken_Access_Control (accessed Apr. 25, 2026)

[MC2] Edge Delta Team. "6 Most Common Observability Mistakes." edgedelta.com. https://edgedelta.com/company/knowledge-center/most-common-observability-mistakes (accessed Apr. 26, 2026)

[JC1] A05 Injection - OWASP Top 10:2025 owasp.org. https://owasp.org/Top10/2025/A05_2025-Injection (accessed Apr. 20, 2026)

[JC2] A07 Authentication Failures - OWASP Top 10:2025 owasp.org. https://owasp.org/Top10/2025/A07_2025-Authentication_Failures (accessed Apr. 20, 2026)

[KS1] MITRE. "CWE-327: Use of a Broken or Risky Cryptographic Algorithm." cwe.mitre.org. https://cwe.mitre.org/data/definitions/327.html (accessed Apr. 27, 2026)

[KS2] MITRE. "CWE-345: Insufficient Verification of Data Authenticity." cwe.mitre.org. https://cwe.mitre.org/data/definitions/345.html (accessed Apr. 27, 2026)

[KS3] OWASP. "Application Security Verification Standard (ASVS) 11.3.3." owasp.org. https://owasp.org/www-project-application-security-verification-standard/ (accessed Apr. 27, 2026)

[KS4] OWASP. "OWASP Proactive Controls – C5: Validate All Inputs." owasp.org. https://owasp.org/www-project-proactive-controls/ (accessed Apr. 27, 2026)

[KS5] OWASP. "A02:2025 – Security Misconfiguration." owasp.org. https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/ (accessed Apr. 27, 2026)

[KS6] MITRE. "CWE-284: Improper Access Control." cwe.mitre.org. https://cwe.mitre.org/data/definitions/284.html (accessed Apr. 27, 2026)

SAST – SEMGREP

[JC3] Source code analysis tools - OWASP Top 10:2025 semgrep.dev. https://semgrep.dev/docs/getting-started/cli#scan-your-project (accessed Apr. 24, 2026)

[JC4] The ten most critical web application security risks semgrep.dev. https://owasp.org/www-community/Source_Code_Analysis_Tools (accessed Apr. 24, 2026)

[JC5] Quickstart: Semgrep - OWASP Top 10:2025 semgrep.dev. https://owasp.org/Top10/2025/0x00_2025-Introduction/ (accessed Apr. 24, 2026)

[KS7] MITRE. "CWE-89: Improper Neutralization of Special Elements used in an SQL Command." cwe.mitre.org. https://cwe.mitre.org/data/definitions/89.html (accessed Apr. 27, 2026)

[KS8] OWASP. "A05:2025 – Injection." owasp.org. https://owasp.org/Top10/2025/A05_2025-Injection/ (accessed Apr. 27, 2026)

[MC3] MITRE. "CWE-94: Improper Control of Generation of Code ('Code Injection')." cwe.mitre.org. https://cwe.mitre.org/data/definitions/94.html (accessed Apr. 27, 2026)

[MC4] OWASP. "A03:2021 – Injection." owasp.org. https://owasp.org/Top10/2021/A03_2021-Injection/ (accessed Apr. 27, 2026)

[MC5] MITRE. "CWE-798: Use of Hard-coded Credentials." cwe.mitre.org. https://cwe.mitre.org/data/definitions/798.html (accessed Apr. 27, 2026)

[MC6] OWASP. "A07:2021 – Identification and Authentication Failures." owasp.org. https://owasp.org/Top10/2021/A07_2021-Identification_and_Authentication_Failures/ (accessed Apr. 27, 2026)

DAST – ZAP

[MC7] OWASP. "A1:2017-Injection." owasp.org. https://owasp.org/www-project-top-ten/2017/A1_2017-Injection# (accessed Apr.26, 2026)

[MC8] Mozilla. "Content Security Policy (CSP)." developer.mozilla.org. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP (accessed Apr. 26, 2026)

[MC9] PortSwigger. "Same-origin policy (SOP)." portswigger.net. https://portswigger.net/web-security/cors/same-origin-policy (accessed Apr. 26, 2026)

[MC10] PortSwigger. "Clickjacking (IU redressing)." portswigger.net. https://portswigger.net/web-security/clickjacking (accessed Apr. 26, 2026)